S32K3XX
PINS & CLOCKS

Automotive Systems & Applications Engineering Team
GPIS-BL, Automotive Processing

SECURE CONNECTIONS
FOR A SMARTER WORLD



PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



AGENDA

1. S32K3 Clocking Overview
2. Lab: S32DS Clocks & Peripherals ConfigTools.
3. S32K3 Signal Multiplexing (SIUL2) Overview
4. Lab: S32DS Pins ConfigTools.

S32K3 Clocking Overview

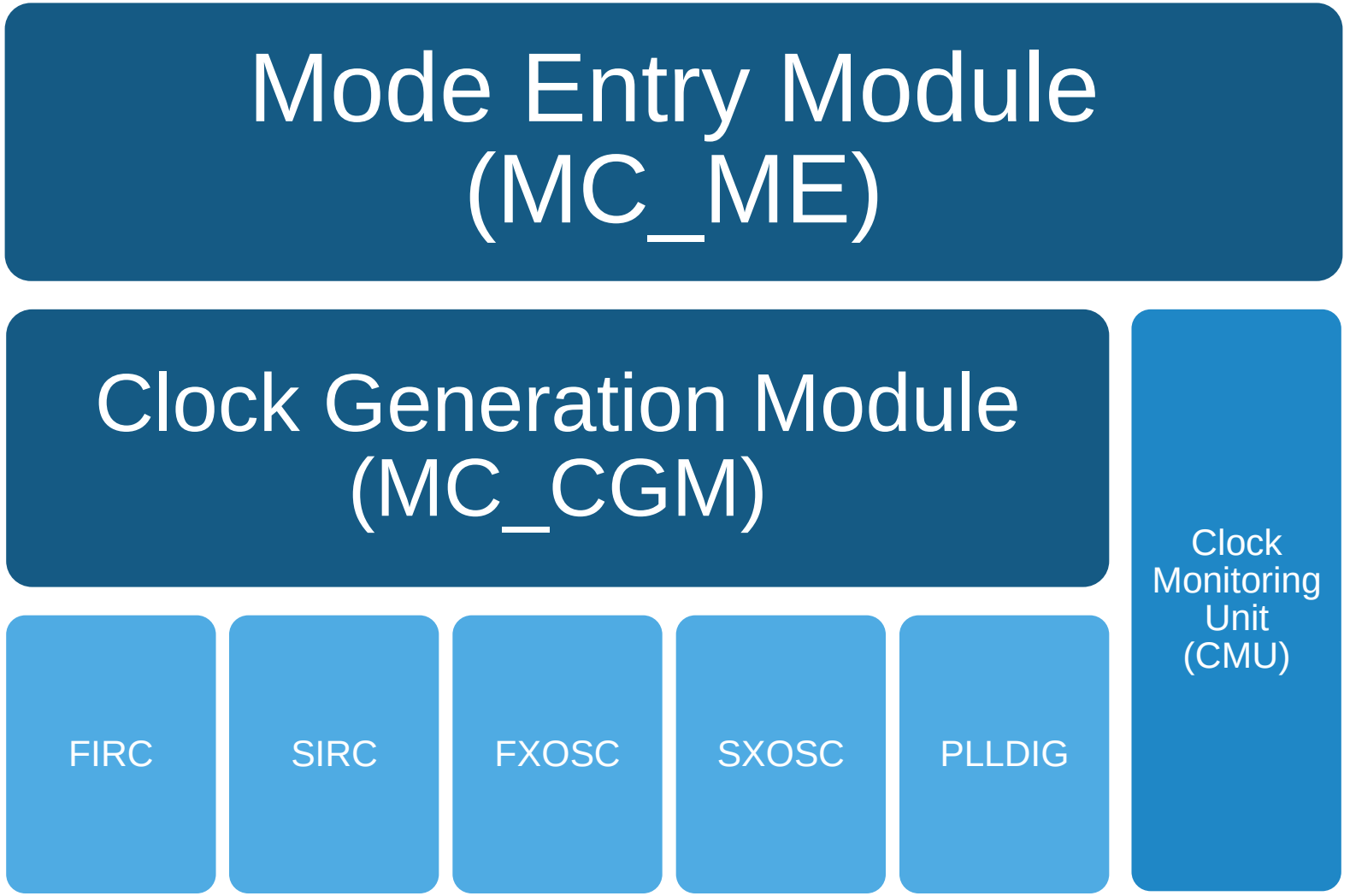


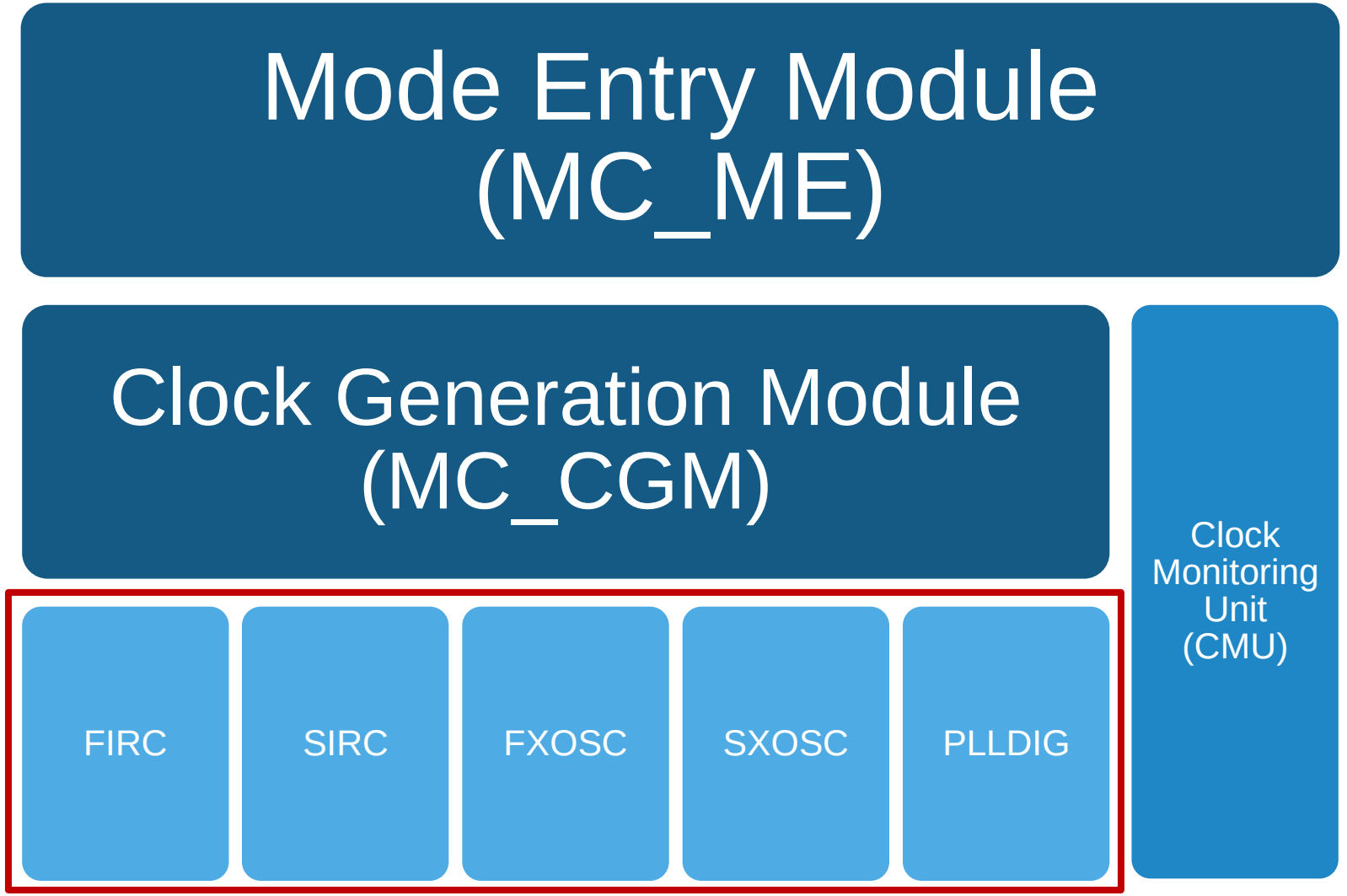
SECURE CONNECTIONS
FOR A SMARTER WORLD



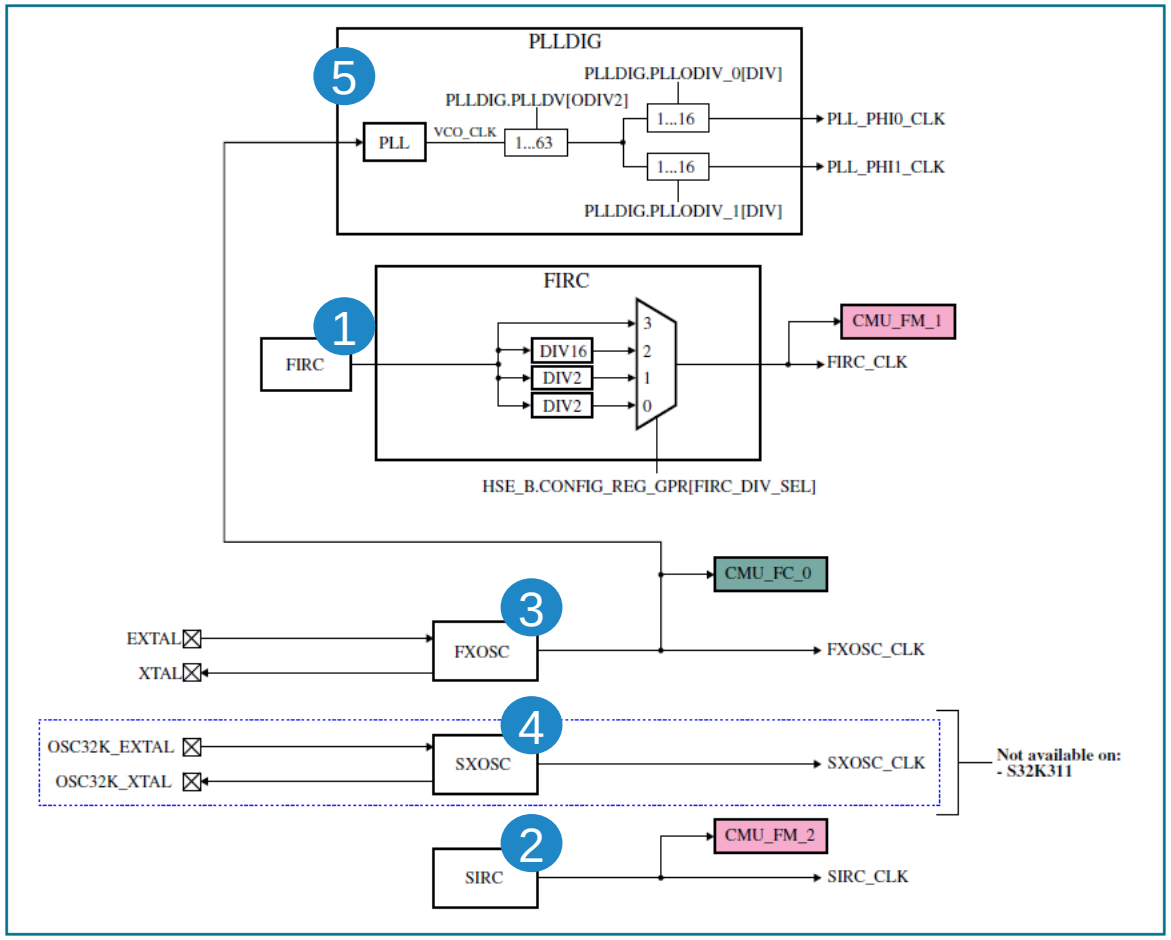
PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

CLOCK SOURCES AND MONITORING UNITS



FIRC reference
FXOSC reference

CLOCK SOURCES

1. Fast Internal RC oscillator (FIRC) – Default.
2. Slow Internal RC oscillator (SIRC)
3. Fast External Crystal oscillator (FXOSC)
4. Slow External Crystal oscillator (SXOSC)
5. Phase-Locked Loop (PLL)

CLOCK MONITORING UNITS

1. Frequency Check (CMU_FC).
2. Frequency Meter (CMU_FM).

CLOCK SOURCES

Clock	Uses	Default	STANDBY
FIRC (48 MHz)	✓ Boot clock (on power up & after any reset event and standby exit). ✓ SIUL2 filter clock. ✓ Safe clock for FCCU and FOSU.	ON	Optional
SIRC (32 KHz)	✓ SWT & POR_WDG clock source. ✓ Safe clock along with FIRC.	ON	Optional
SXOSC (32.768 KHz)	✓ SXOSC is not affected by functional reset. ✓ RTC source for operation across functional reset.	OFF	Optional
FXOSC (8-40 MHz)	✓ Reference clock source for PLL. ✓ Communication modules (FlexCAN, EMAC, QuadSPI, and so on).	OFF	Optional
PLL (48-320 MHz)	✓ Optional system clock source (in high performance applications). ✓ Communication modules (FlexCAN, EMAC, QuadSPI, and so on).	OFF	Not supported

FIRC & SIRC

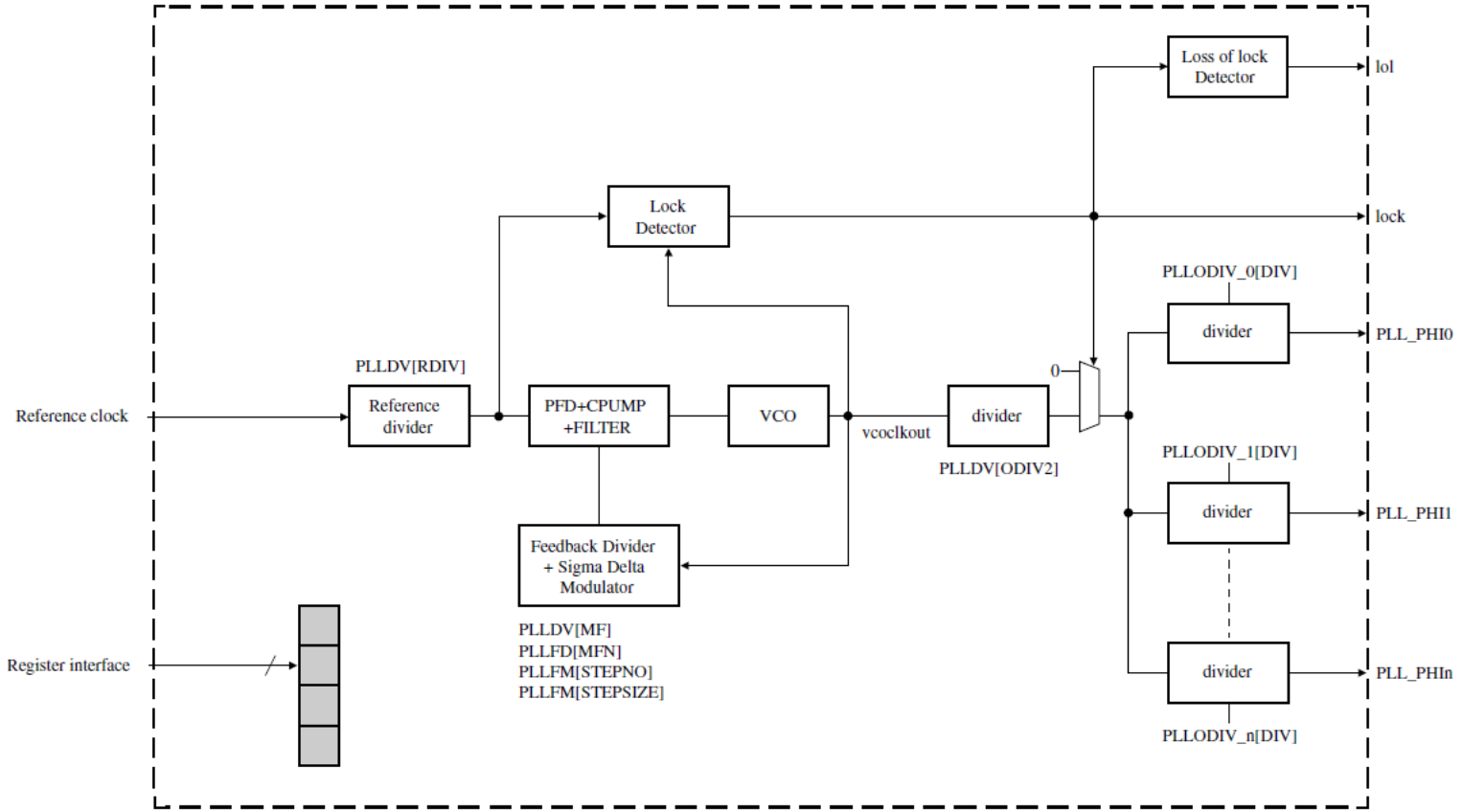
- Always enabled in RUN mode.
- Status registers provide the current operating state
  a. ON and stable
  b. OFF or not stable
- Safety clocks for safety relevant applications for modules responsible to detect and react to incorrect chip operation.

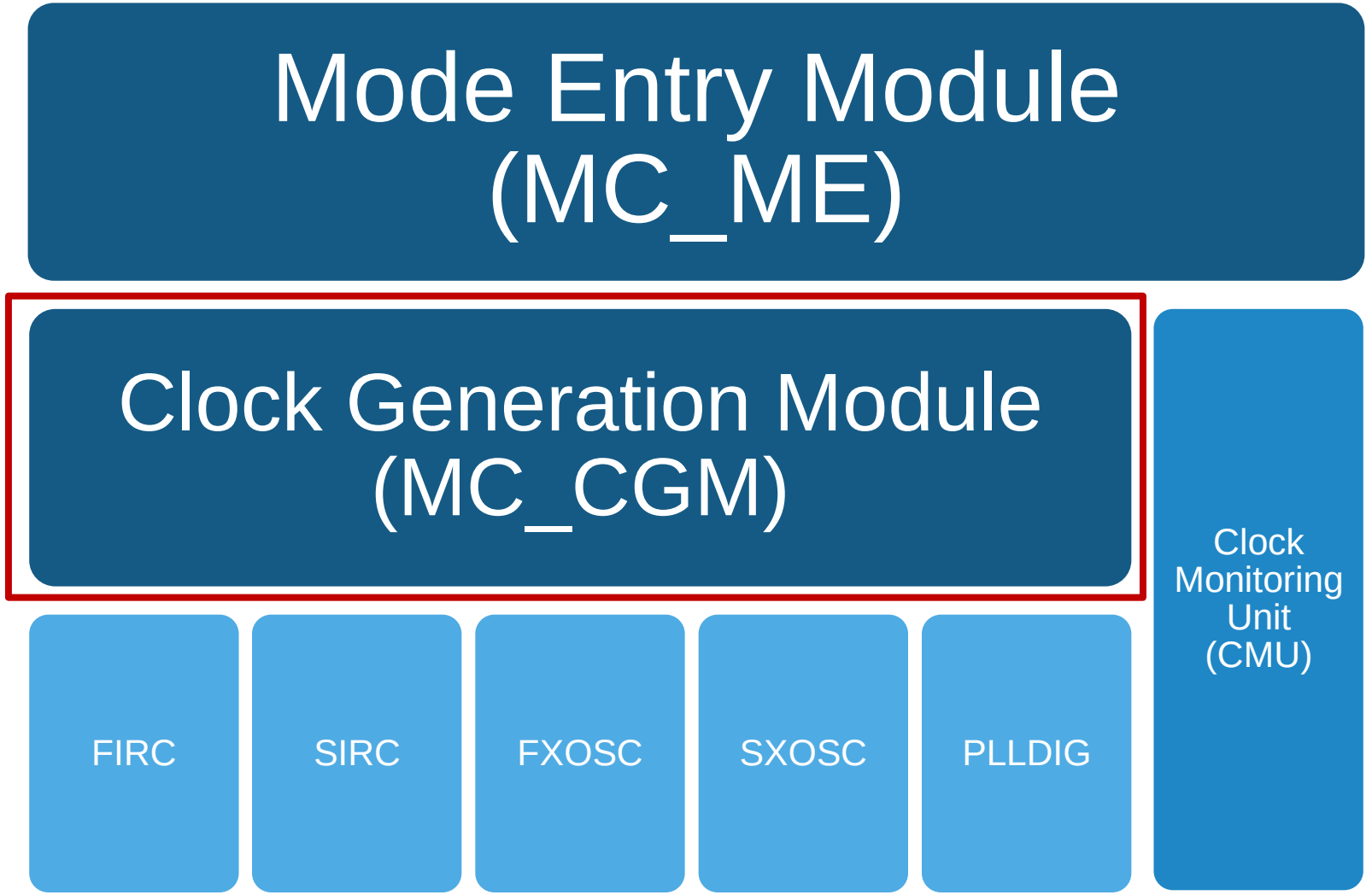
FXOSC & SXOSC

- Clock gating by the MC_ME.
- Status register showing current state
  a. ON and providing a stable clock.
  b. OFF or its output clock is not stable.
- Support
  a. Crystal input mode
  b. Bypass mode
- Configurable stabilization counter value.
- **FXOSC**: Configurable amplifier transconductance.

User interface to control the PLL system.

- Programmable frequency modulation.
- Multiple integer dividers on PLL outputs.
- Lock detection circuitry reports when the PLL achieves frequency lock.
- Continuous monitoring of lock status to report Loss of Lock (LOL) condition.

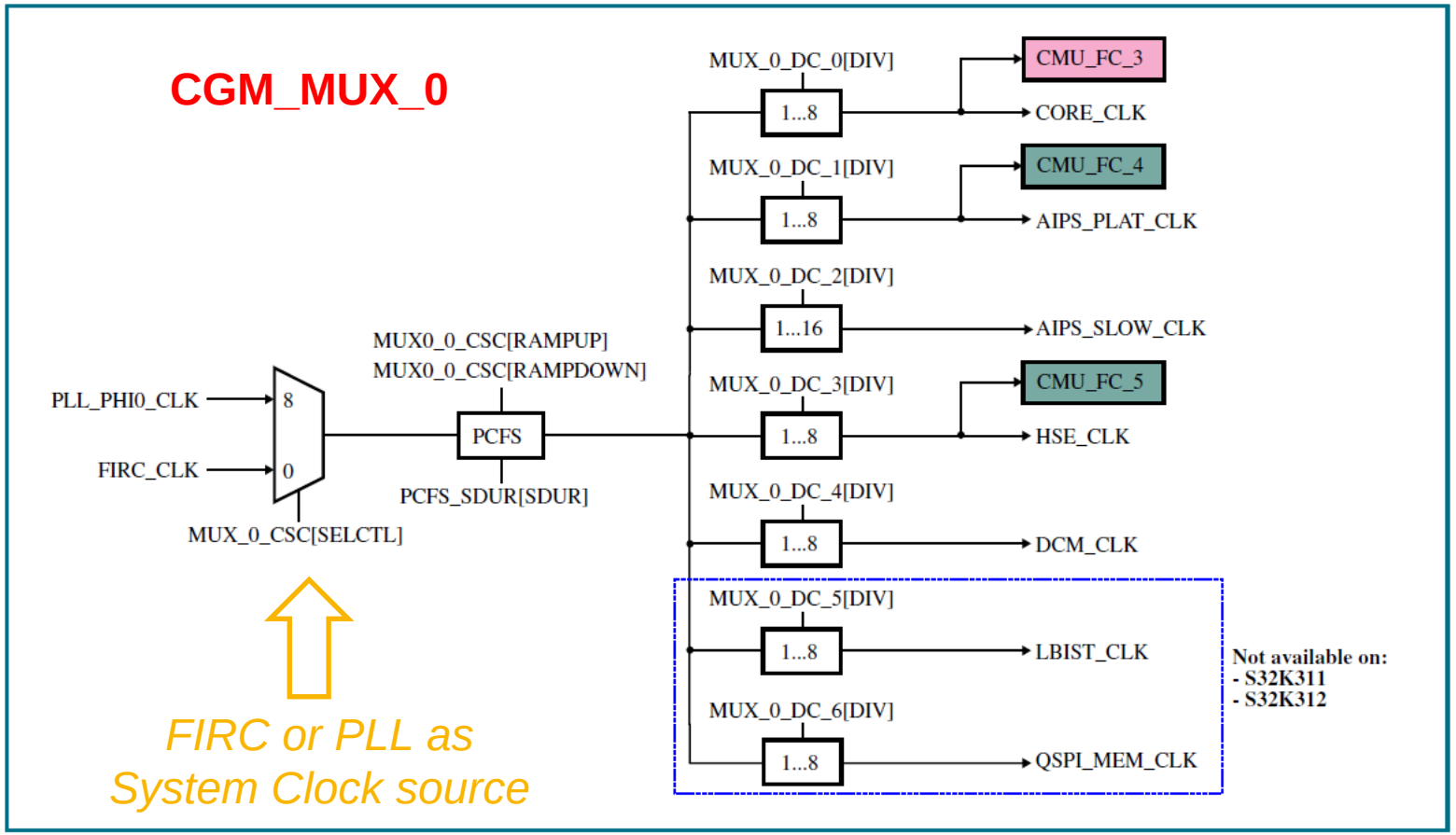



Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

Generates reference clocks for all the chip blocks.

Features:

- SW-configurable multiplexers and dividers for on-chip resources.
- Progressive Clock Frequency Switching (PCFS) to minimize the impact of a sudden power consumption change when switching clock sources.
- Common trigger to update all clock dividers within a MUX at the same time
- Glitch-less clock transitions when changing the clock selection at clock multiplexers.

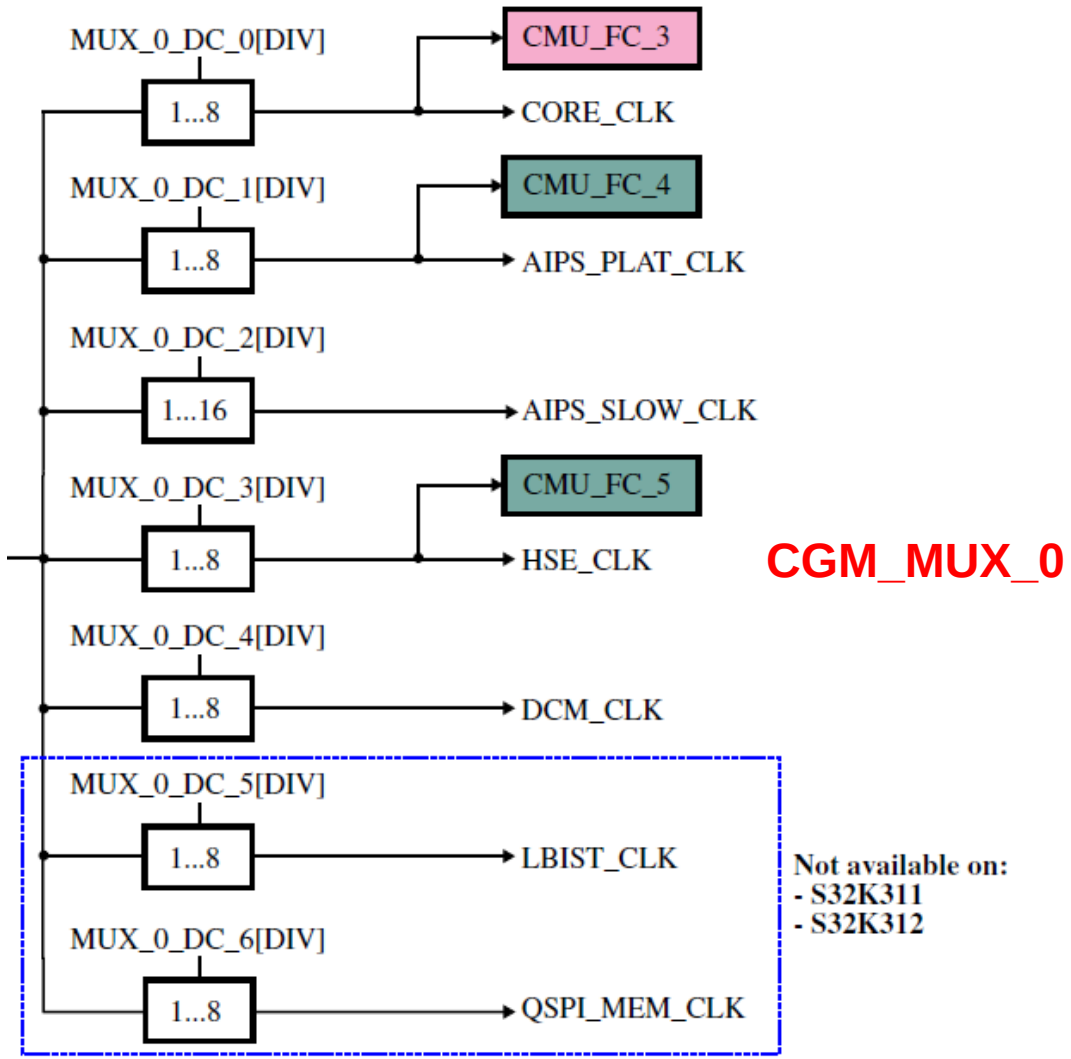


There are 12 clock multiplexers on S32K344: clock **MUX 0 ~ MUX 11**

FIRC reference

FXOSC reference

SYSTEM CLOCKING (CGM_MUX_0)

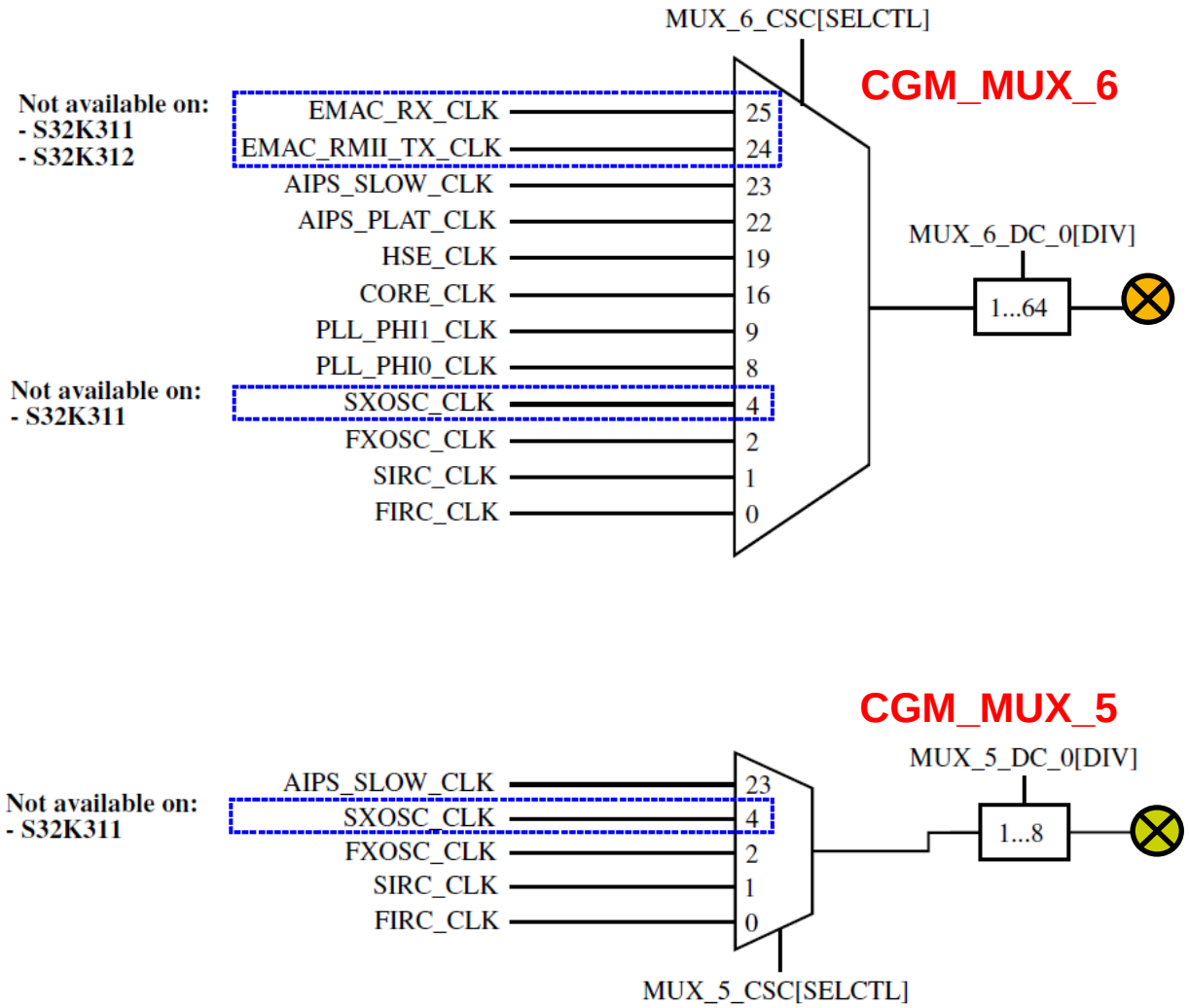


CGM_MUX_0

Node	Clock Source for…
CORE_CLK	Application cores, AXBS, SRAM, PFLASH and AIPS0 (high speed peripherals).
AIPS_PLAT_CLK	Medium speed peripherals (e.g. FlexCAN, LPUART, FlexIO).
AIPS_SLOW_CLK	Slow speed peripherals (e.g. LPI2C, LPSPI1-5, SIUL2).
HSE_CLK	HSE CM0+
DCM_CLK	Device Configuration Module
LBIST_CLK	STCU2 for LBIST operations.
QSPI_MEM_CLK	QSPI_RAM and QSPI_TX blocks.

All clock dividers in MUX_0 are enabled by default after reset.

CLKOUT OVERVIEW



The chip supports two CLKOUT pins for allowing the view of some internal clocks.

- **CLKOUT_RUN (CGM_MUX_6)**

Available during Run mode only.

PTB5, PTD10, PTD14

- **CLKOUT_STANDBY (CGM_MUX_5)**

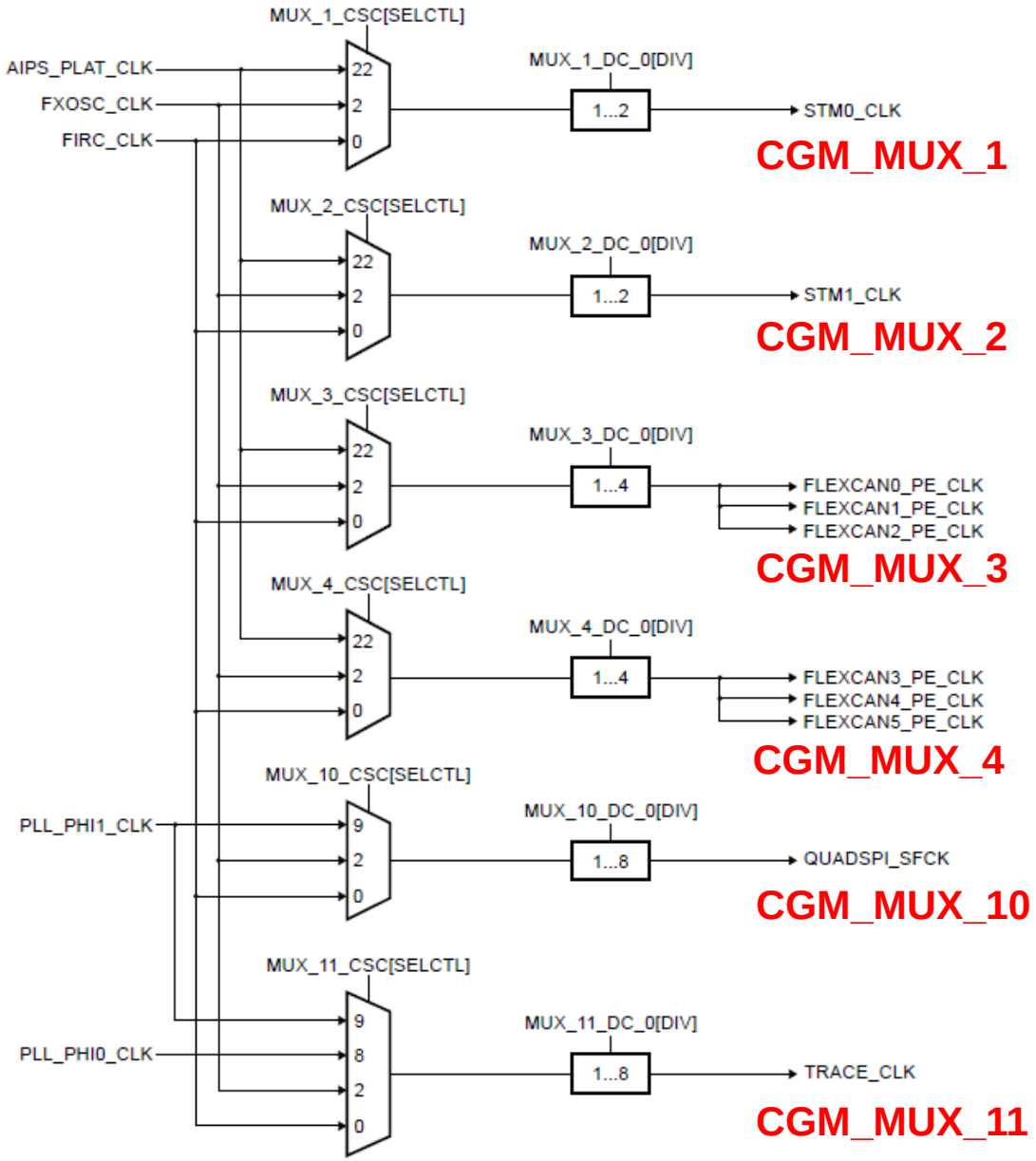
Available during both Run and Standby modes.

(Its registers are latched while the chip enters Standby mode and are reset in Standby exit sequence. Therefore, the CLKOUT_STANDBY signal needs to be reconfigured on Standby mode exit).

PTA12, PTE10

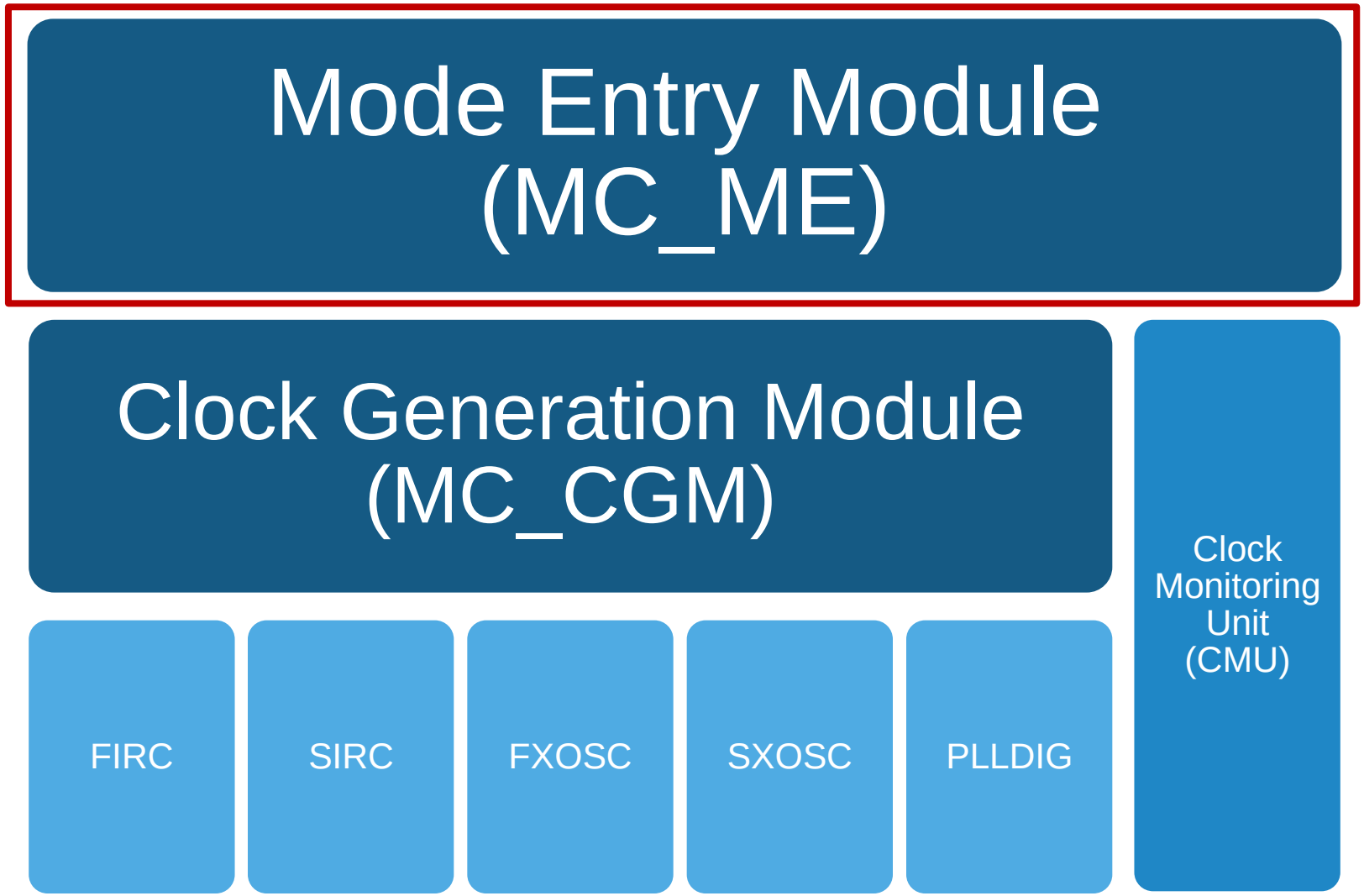
Clock dividers in MUX_5 and MUX_6 are enabled by default.

OTHER CLOCKS…



Node	Clock source for…
STMx_CLK	Both the operating and register interface clocks for the System Timer Modules (STMx).
FLEXCANx_PE_CLK	FlexCANx protocol engine clocks.
QUADSPI_SFCK	QuadSPI serial flash clock.
TRACE_CLK	Trace subsystem blocks.

Clock dividers in these clock MUX are disabled by default.


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

Provides core and peripheral clock gating.

Each IP is assigned to a

- Partition (**PRTN**) and a
- Collection of Functional Blocks (**COFB**).

And has its own clock enabler (**CLKEN**).

Table 185. MC_ME Partition Peripheral mapping and Clock Control

AIPS	Peripheral description	MC_ME COFB control register	MC_ME peripheral control register	MC_ME peripheral slot no. in partition	MC_ME COFB present	MC_ME CLKEN present	MC_ME Default CLKIN	On Platform	IPS SLOT NUMBER
0	Reserved	PRTN0_COFB0_CLKEN[REQ0]	0	0	0	0	0	YES	0
0	Reserved	PRTN0_COFB0_CLKEN[REQ1]	1	1	0	0	0	YES	1
0	Reserved	PRTN0_COFB0_CLKEN[REQ2]	2	2	0	0	0	YES	2
0	Reserved	PRTN0_COFB0_CLKEN[REQ3]	3	3	0	0	0	YES	3
0	Reserved	PRTN0_COFB0_CLKEN[REQ4]	4	4	0	0	0	YES	4
0	Reserved	PRTN0_COFB0_CLKEN[REQ5]	5	5	0	0	0	YES	5
0	Reserved	PRTN0_COFB0_CLKEN[REQ6]	6	6	0	0	0	YES	6
0	Reserved	PRTN0_COFB0_CLKEN[REQ7]	7	7	0	0	0	YES	7

S32K3xx_RM

The **MC_ME** module supports a mechanism for starting the pre-configured hardware processes, through a simple key-writing sequence.

Before using a peripheral, turn on its clock.

Step	Description	Register
0	Consider the previous peripheral requirements/sequence.	
1	Program the corresponding clock enabler.	MC_ME . PRTNx_COFBy_CLKEN [**REQz**]
2	Program the corresponding process update.	MC_ME . PRTNx_PUPD [**PCUD**]
3	Enter the key, then the inverted key.	MC_ME . CTL_KEY
4	Check if the HW process has finished.	MC_ME . PRTNx_PUPD [**PCUD**]

SW EXAMPLE: ENABLING PIT0'S CLOCK

0	Programmable Interrupt Timer 0	PRTN0_COFB1_CLKEN[REQ44]	44	44	1	1	1	NO	12
0	Programmable Interrupt Timer 1	PRTN0_COFB1_CLKEN[REQ45]	45	45	1	1	0	NO	13

S32K3xx_RM

```
// Enabling clock for the PIT0 (REQ44 in COFB1 of PRTN0)
MC_ME->PRTN0_COFB1_CLKEN |= MC_ME_PRTN0_COFB1_CLKEN_REQ44_MASK;
MC_ME->PRTN0_PUPD        |= MC_ME_PRTN0_PUPD_PCUD_MASK;
MC_ME->CTL_KEY = 0x5AF0; // Enter key
MC_ME->CTL_KEY = 0xA50F; // Enter inverted key

// Wait the HW process to finish
while(MC_ME->PRTN0_PUPD & MC_ME_PRTN0_PUPD_PCUD_MASK){}
```

It is possible to enable/disable multiple peripherals/cores in the same HW process.

The application core clocks are gated by individual MC_ME core clock enable bits.
For HSE core, there is no clock control.

Step	Description	Register
0	Consider the previous core requirements/sequence.	
1	(Init) Write the application core boot address	MC_ME . PRTN0_COREx_ADDR
	(Shut down) Read the application core WFI status	MC_ME . PRTN0_COREx_STAT[**WFI**]
2	Program the corresponding core clock enabler.	MC_ME . PRTN0_COREx_PCONF [**CCE**]
3	Program the corresponding process update.	MC_ME . PTRN0_COREx_PUPD [**CCUPD**]
4	Enter the key, then the inverted key.	MC_ME . CTL_KEY
5	Check if the HW process has finished.	MC_ME . PTRN0_COREx_PUPD [**CCUPD**]

SW EXAMPLE: DISABLING CM7_1'S CLOCK

Serial No.	Core	MC_ME partition	MC_ME Clock Enable Register Bit
1	CM7_0 (Cortex M7 0)	0	MC_ME.PRTN0_CORE0_PCONF[CCE]
2	CM7_1 (Cortex M7 1)	0	MC_ME.PRTN0_CORE1_PCONF[CCE]

S32K3xx_RM

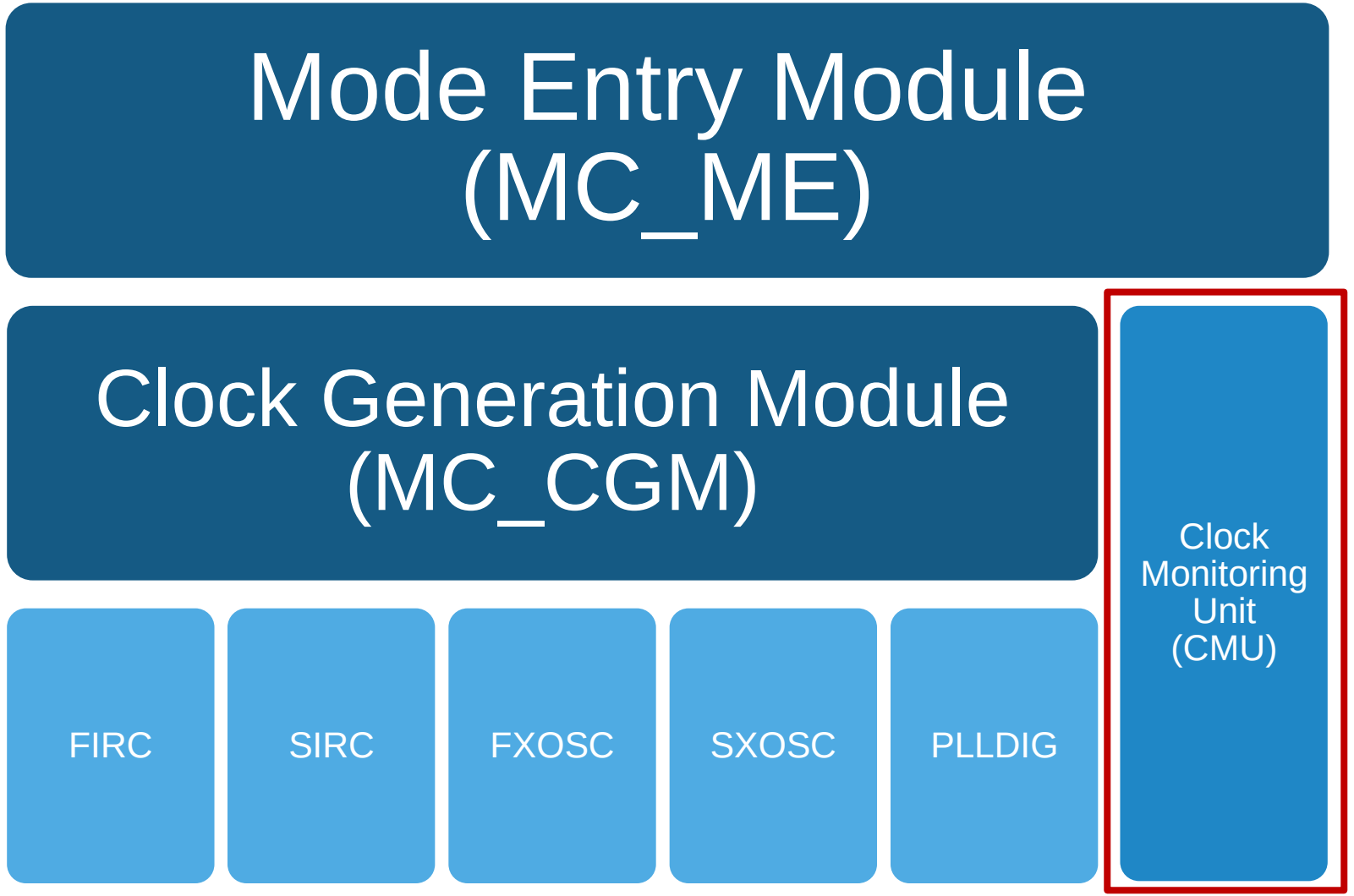


```
// Wait for CM7_1 to execute WFI
while(!(MC_ME->PRTN0_CORE1_STAT & MC_ME_PRTN0_CORE1_STAT_WFI_MASK)) { }

// Disabling clock for CORE1 (CM7_1)
MC_ME->PRTN0_CORE1_PCONF &= MC_ME_PRTN0_CORE1_PCONF_CCE_MASK;
MC_ME->PRTN0_CORE1_PUPD  |= MC_ME_PRTN0_CORE1_PUPD_CCUPD_MASK;
MC_ME->CTL_KEY = 0x5AF0; // Enter key
MC_ME->CTL_KEY = 0xA50F; // Enter inverted key

// Wait the HW process to finish
while(MC_ME->PRTN0_CORE1_PUPD & MC_ME_PRTN0_CORE1_PUPD_CCUPD_MASK) {}
```

Assumptions:
CM7_0 is running this code,
Device is not in Lockstep,
CM7_1 has already executed WFI.


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

- This block consist of six monitors reporting malfunctions in the clocking system.

CMU	Type (Check / Meter)	Monitored or metered clock	Failure reaction
0	**FC** (Precision over and under frequency).	FXOSC_CLK	Destructive RST or Interrupt
1	**FM** (Freq. measurement periodically triggered by SW).	FIRC_CLK	Interrupt
2	**FM** (Freq. measurement periodically triggered by SW).	SIRC_CLK	Interrupt
3	**FC** (Precision over and under frequency).	CORE_CLK	Destructive RST
4	**FC** (Precision over and under frequency).	AIPS_PLAT_CLK	Destructive RST
5	**FC** (Precision over and under frequency).	HSE_CLK	Destructive RST

FIRC reference
FXOSC reference

Lab:
S32DS Clocks & Peripherals ConfigTools



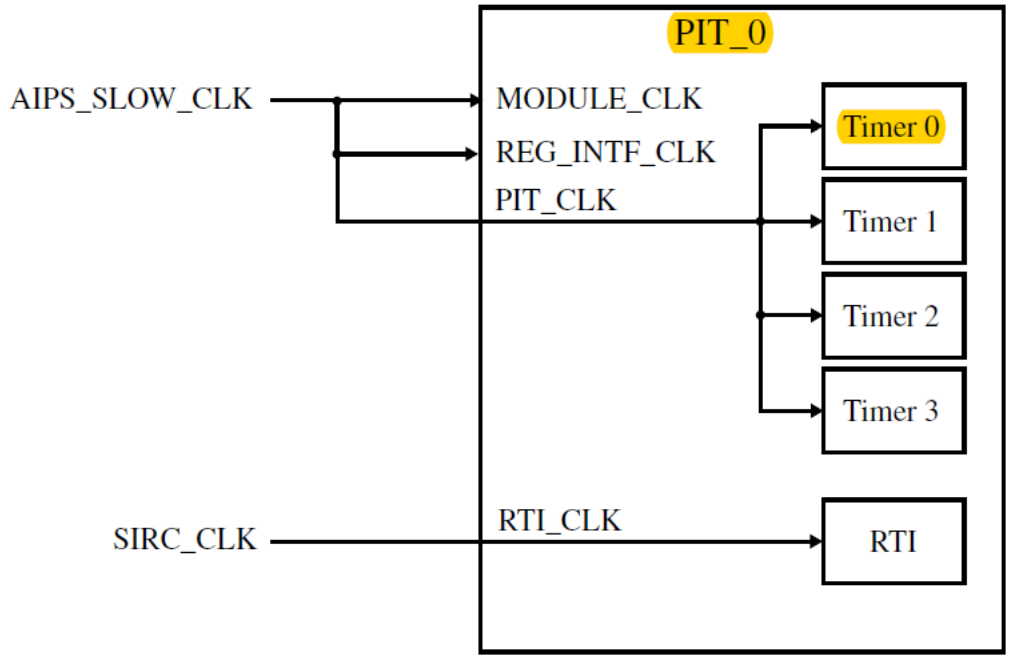
SECURE CONNECTIONS
FOR A SMARTER WORLD



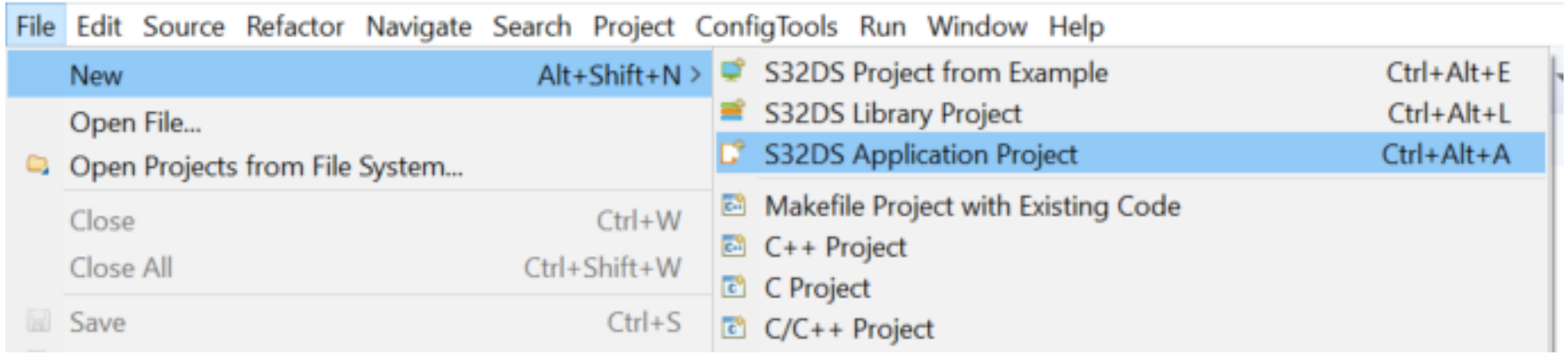
PUBLIC

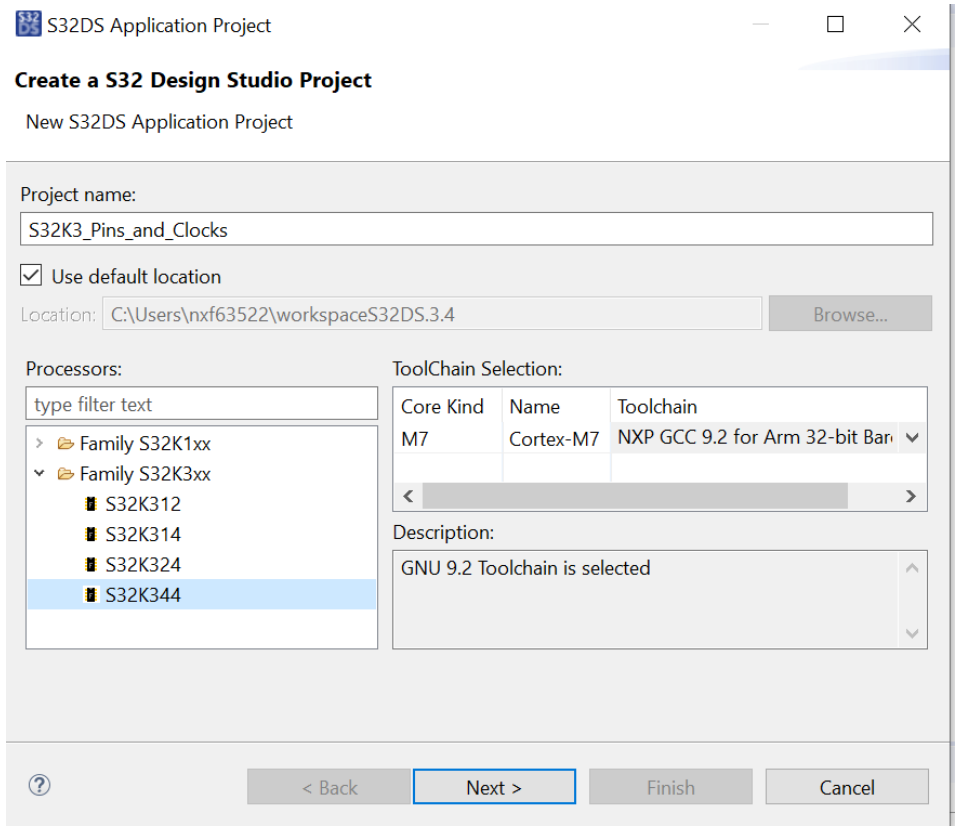
NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.

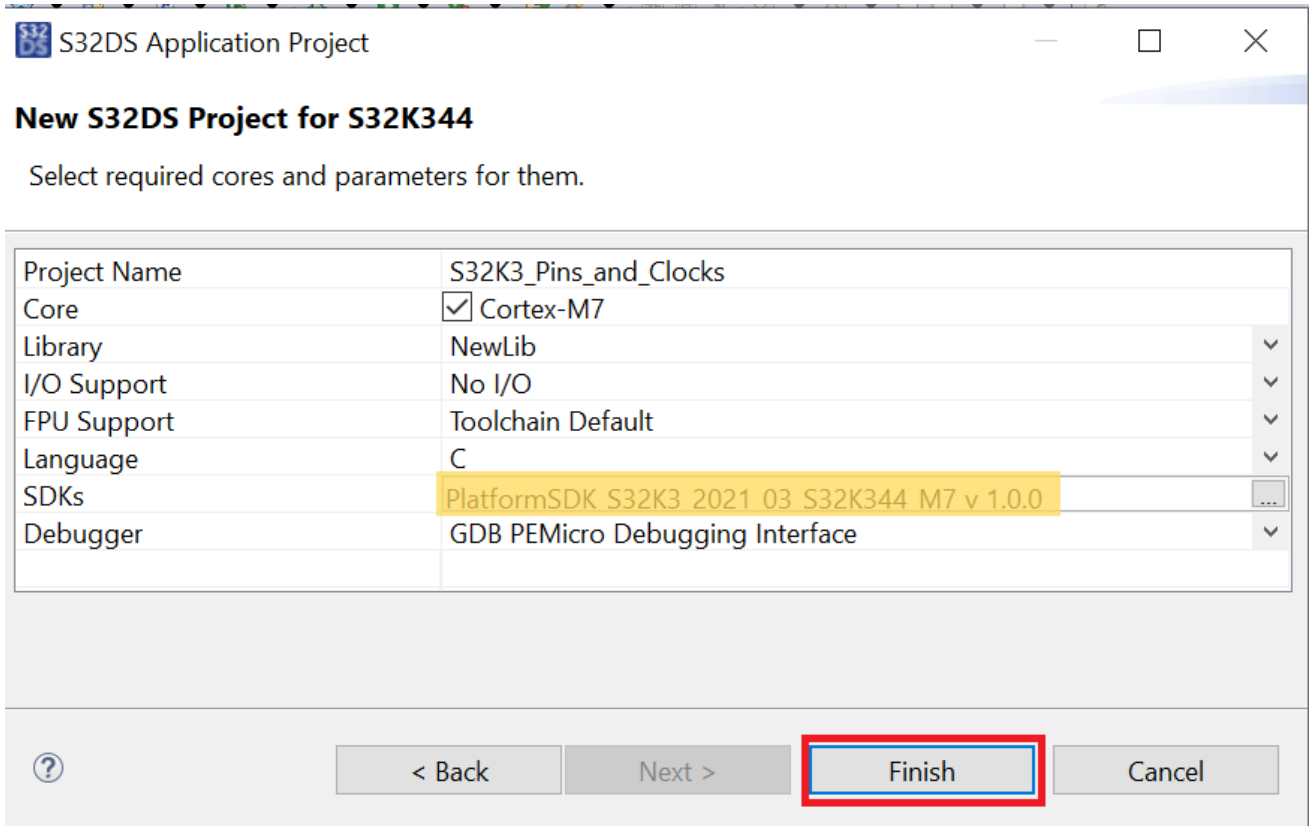
- Become familiar with the **S32DS** environment.
- Look at the S32DS Clocks & Peripherals **ConfigTools**.
- Create an application where **PIT_0 (CH0)** generates an interrupt every second (clocked by the AIPS_SLOW_CLK @ 40 MHz).



- Start **S32DS** with desired workspace.
- File -> New -> **S32DS Application Project.**
- Enter a **Project name**, select the **Target Processor** (S32K344). Click **Next**.
- Select the **Debugger** to use (PEMicro) and latest **SDK**.
- Click **Finish**.

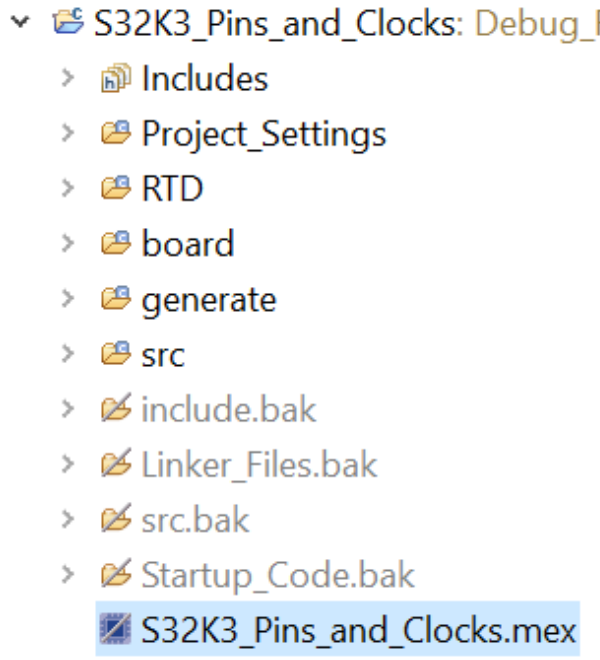


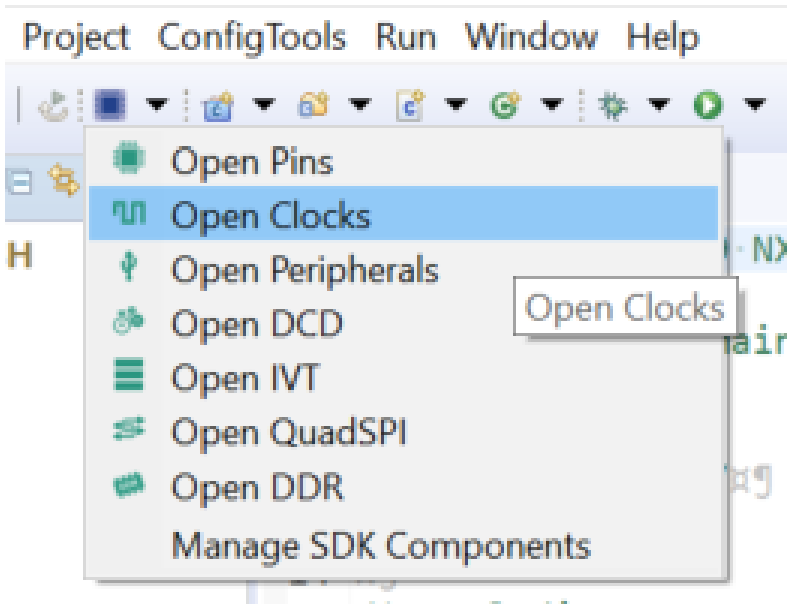




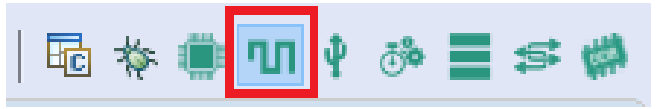
Notice that the SDK could change depending on the RTD version. Select the latest release.

- Expand your project and double click on the *<Project name>.mex* file.
- **Or** click on the “Open S32 Configuration” on the top bar and select “**Open Clocks**”.

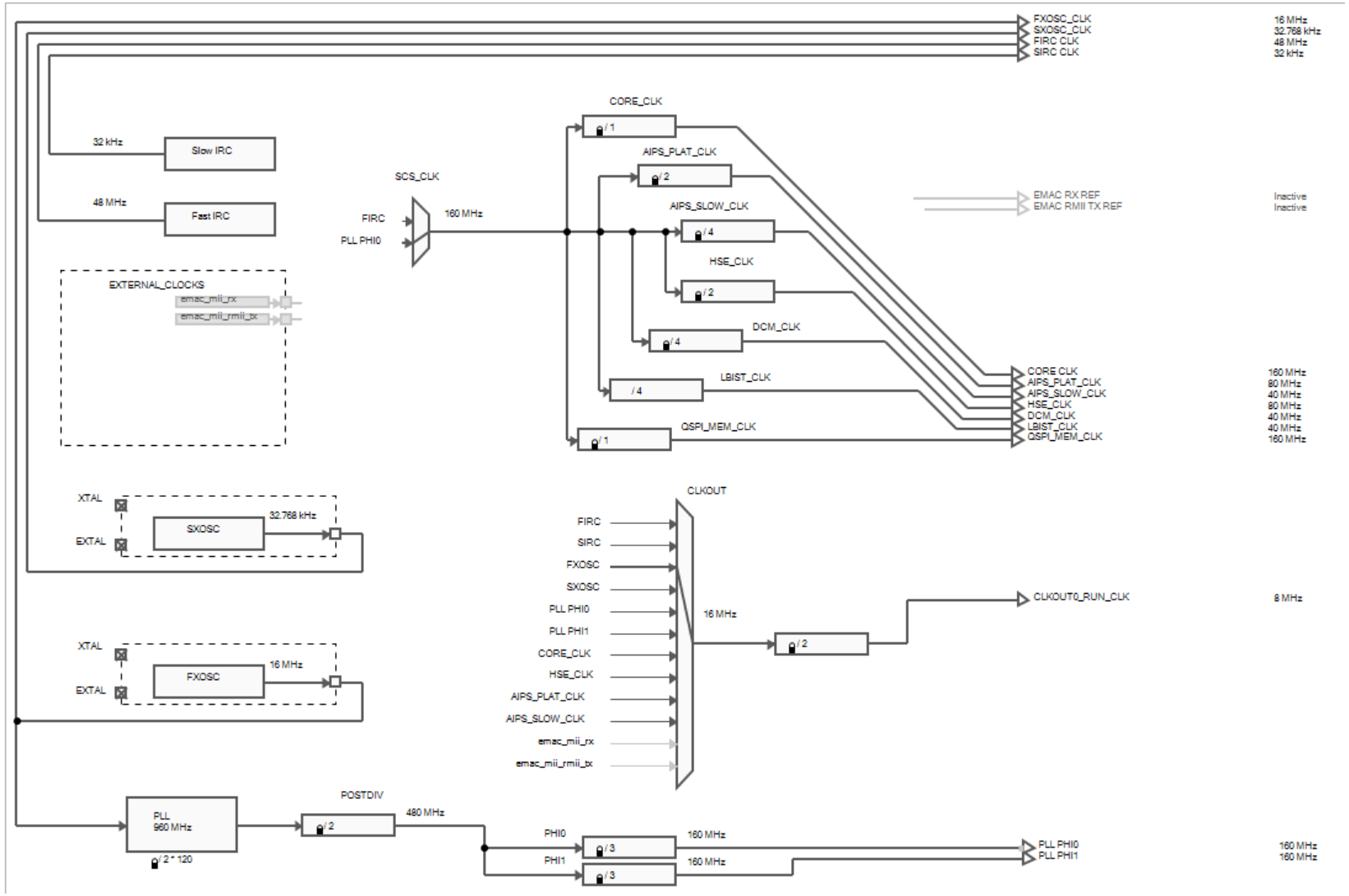




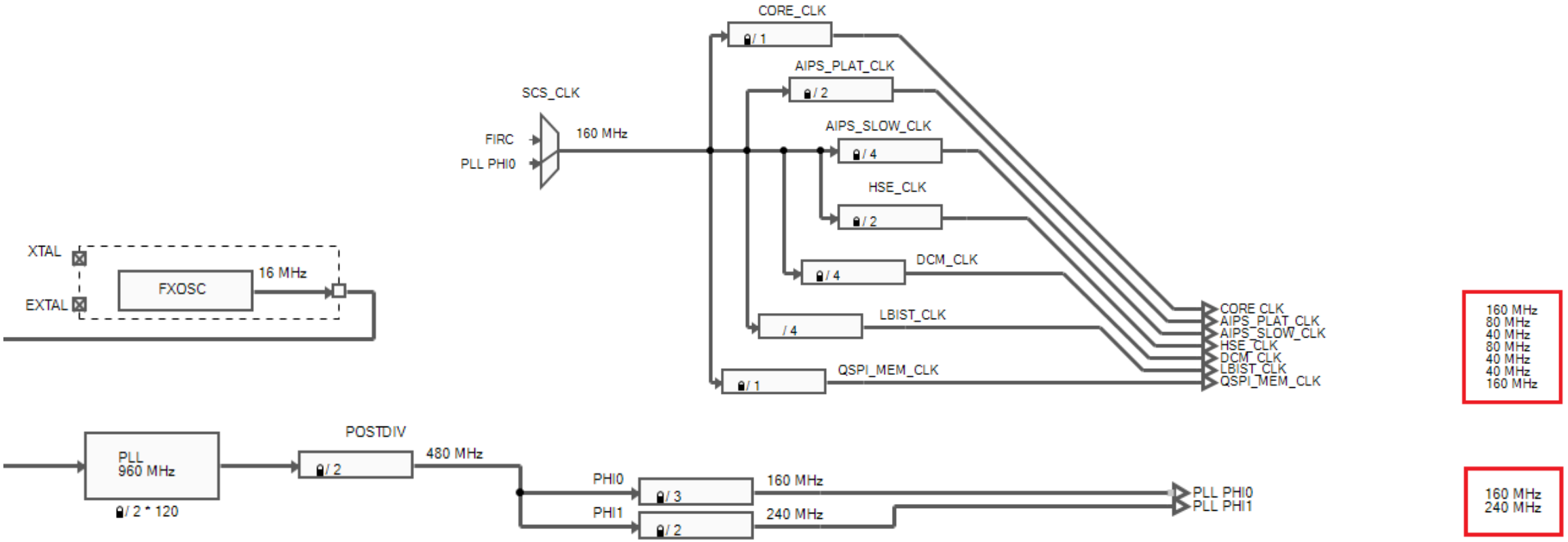
- Make sure, you are in the **Clocks tab** (top right bar).



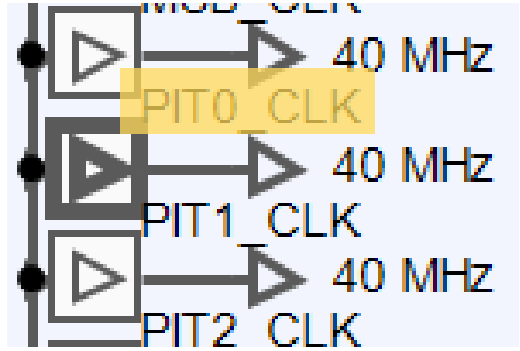
- In the **Clocks Diagram** view, you will see the complete Clocking tree of your device.



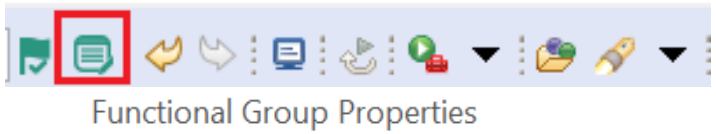
Let's configure the Option A (High Performance mode) as in the Reference Manual (RM), using the 16 MHz crystal from the EVB.



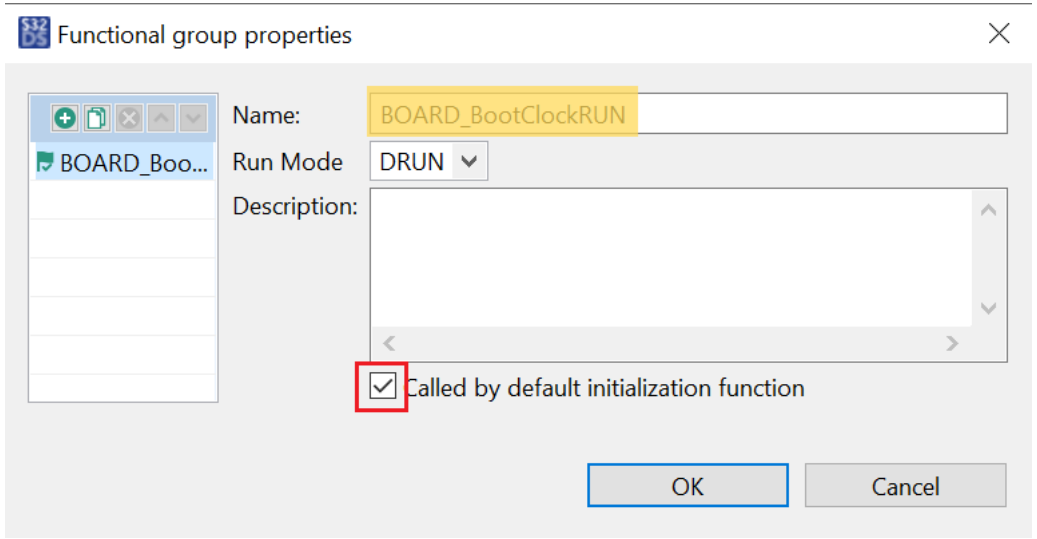
- Scroll down in the Diagram View and enable the PIT0_CLK (clocked by AIPS_SLOW_CLK).



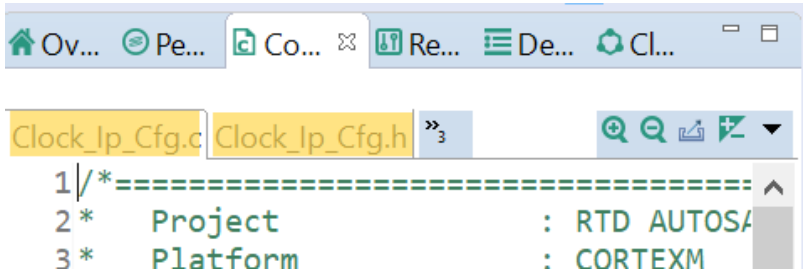
- Open the **Functional Group Properties**



- And check the **Called by default…** Option.



- In the **Code Preview** tab, you can see these files



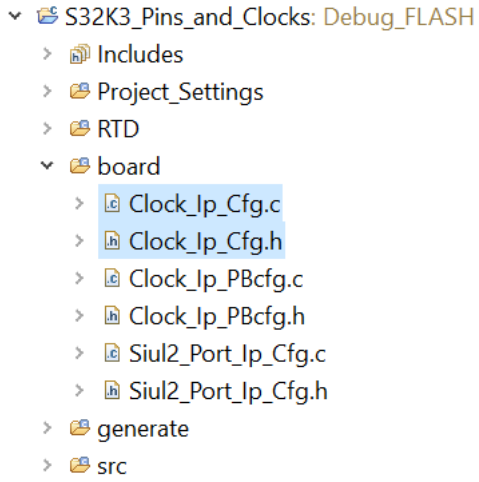
- To add them into your application project, click the **Update Code** button.



- Come back to the **C/C++** perspective.

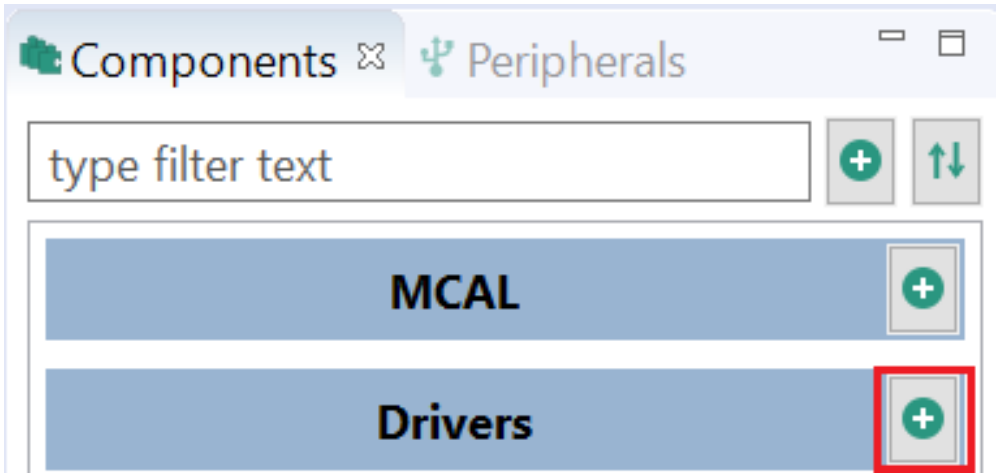


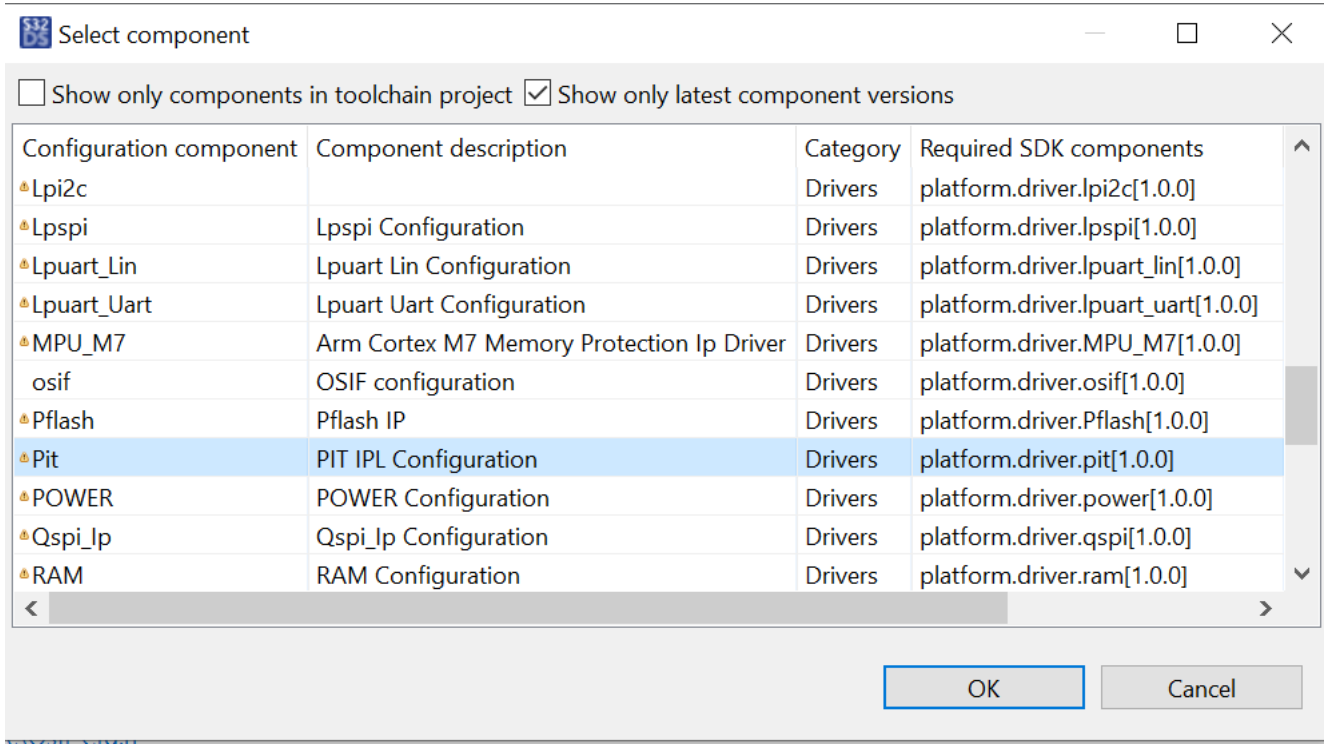
- Code will be available under the **board** folder.



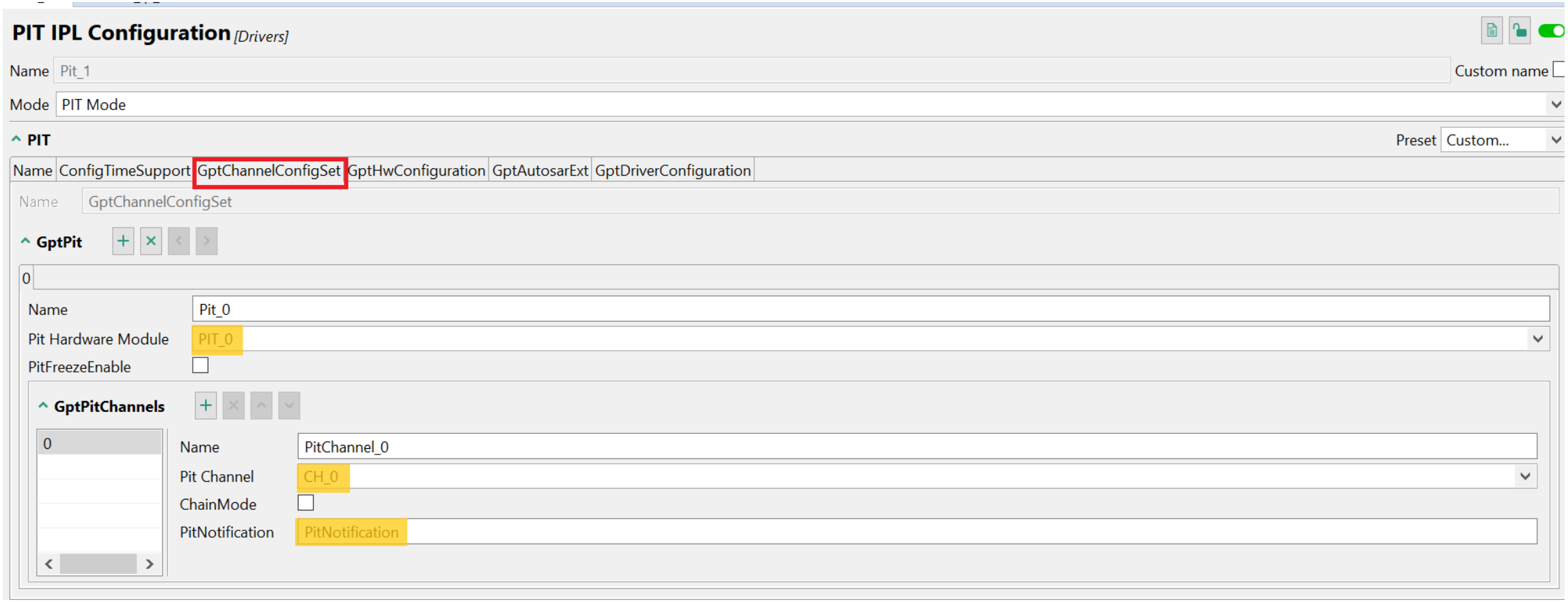
- A similar procedure is required for the PIT config.
- Open Peripherals ConfigTool and add a new Component.
- Scroll down and select the PIT driver.



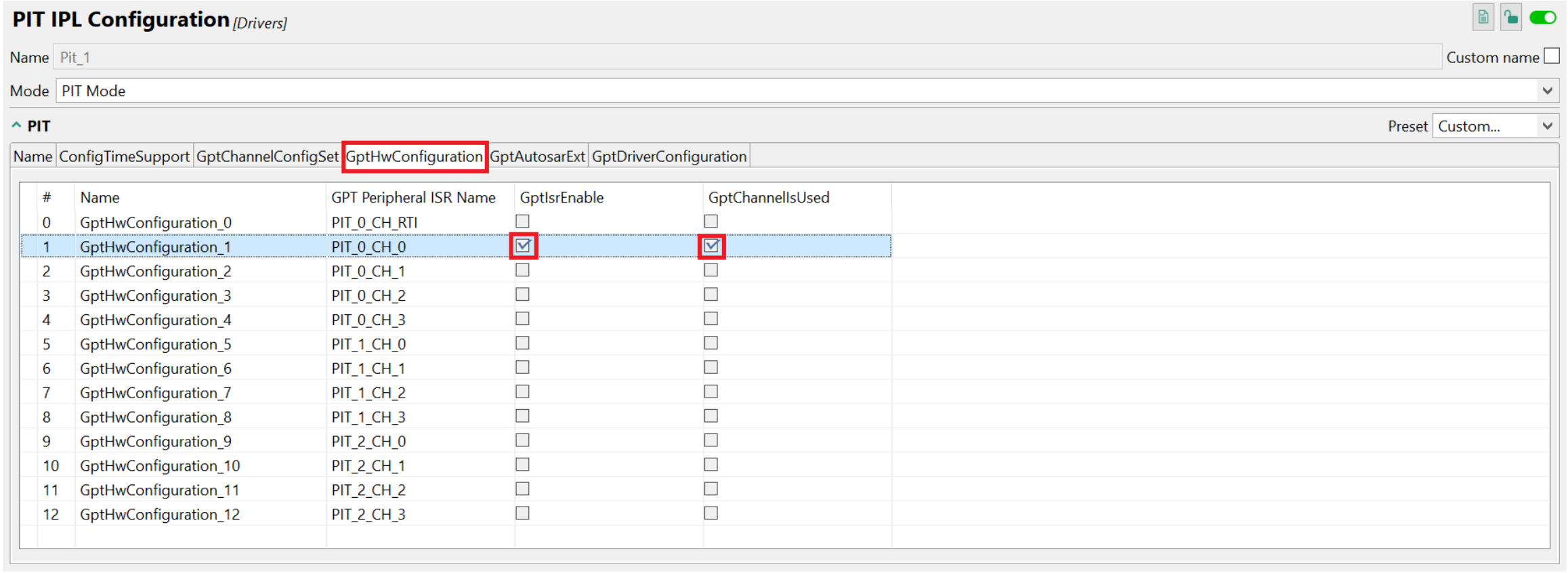




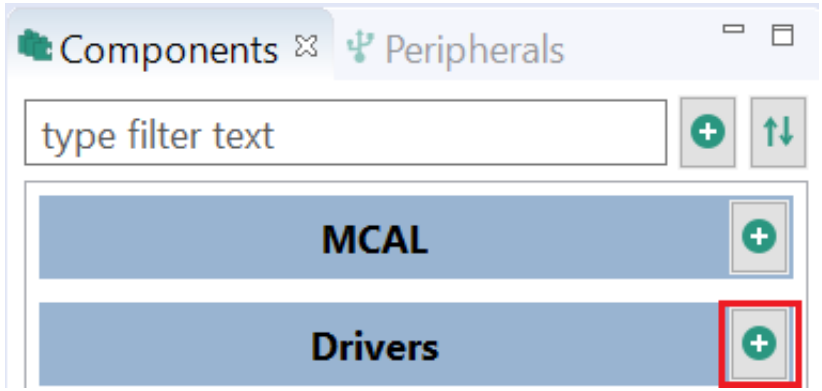
Add a new PIT instance and Channel by using the **“+”** icons, then select the PIT_0 instance, Channel 0, enable Interrupts and name your PIT callback.

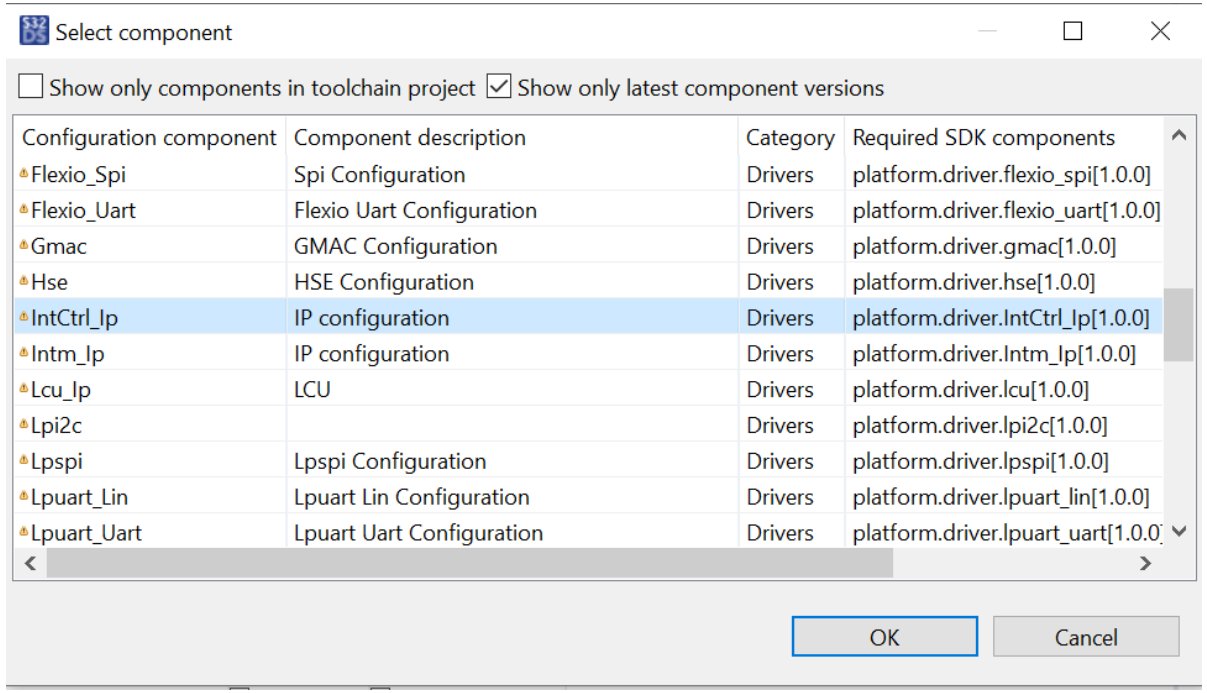


Add a new PIT instance and Channel by using the **“+”** icons, then select the PIT_0 instance, Channel 0, enable Interrupts and name your PIT callback.

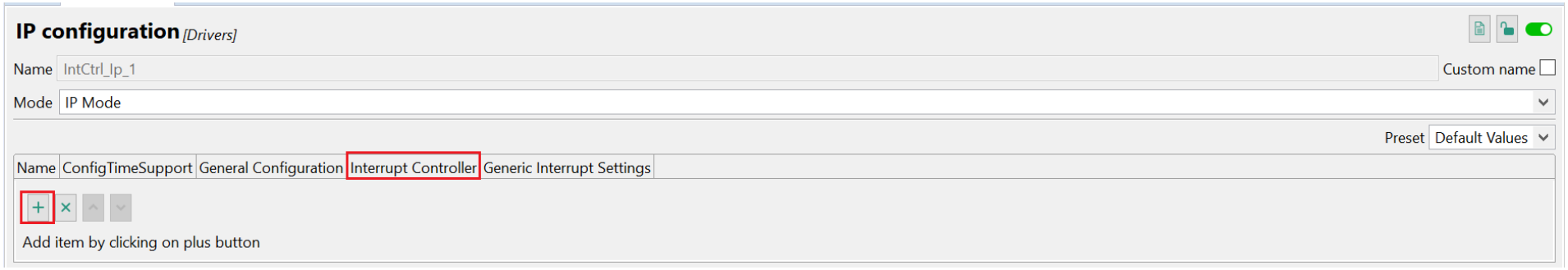


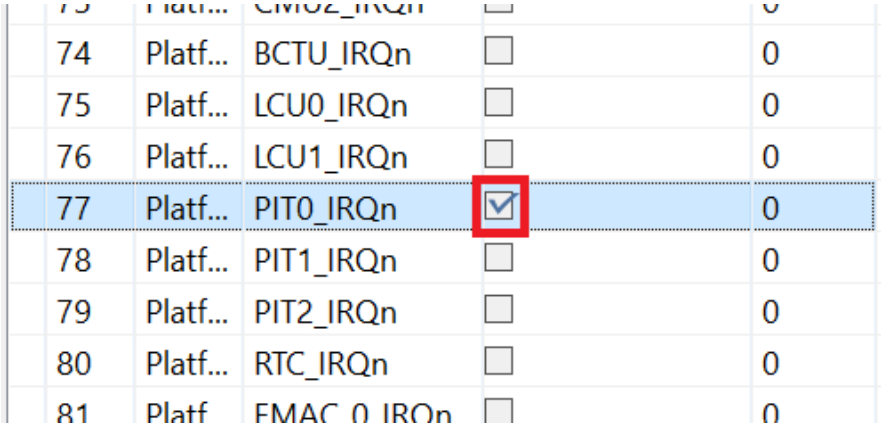
- We also need to add the “IntCtrl_Ip” component for this project.

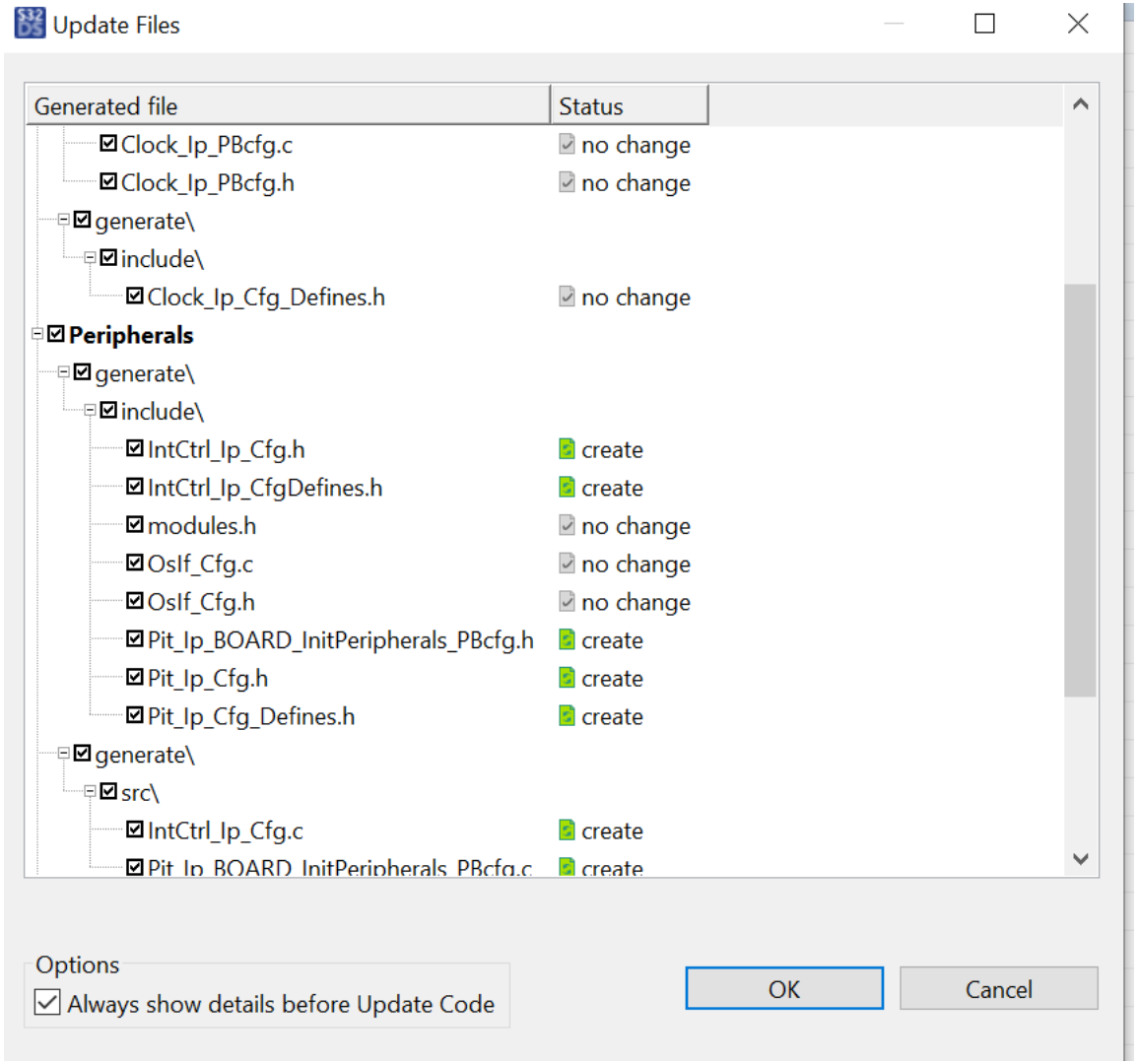




- Then, scroll down and enable the PIT0_IRQn







- Again, we can look at the Code Preview, and add it to the project by clicking the “Update Code” button.



- Open the **main.c** file, include necessary header files.

```
#include "Mcal.h"
#include "Clock_Ip.h"
#include "Pit_Ip.h"
#include "IntCtrl_Ip.h"
```

- And below macros / global variables

```
extern   ISR(PIT_0_ISR);
#define  CH_0        ((uint8)0) /*!< PIT channel used - 0 */
#define  PIT_INST    0U         /*!< PIT instance used - 0 */
#define  PIT_PERIOD  40E6       /*!< PIT timeout period - equivalent to 1sec */
volatile uint32 PitCounter;
```

- Write the PitNotification callback

```
void PitNotification (void)
{
  PitCounter++;
}
```

- And Clocks, Interrupt, PIT configurations within the main function

```
volatile int exit_code = 0;

int main (void)
{
  /* Clocks Configuration */
  Clock_Ip_Init(&Mcu_aClockConfigPB[0]);

  /* Interrupt Configuration */
  IntCtrl_Ip_InstallHandler(PIT0_IRQn, PIT_0_ISR, NULL_PTR);
  IntCtrl_Ip_Init(&IntCtrlConfig_0);
```

APPLICATION CODE

```
 /* PIT0 (CH0) Configuration */
  Pit_Ip_Init(PIT_INST, &PIT_0_InitConfig_PB);
  Pit_Ip_InitChannel(PIT_INST, PIT_0_CH_0);
  Pit_Ip_EnableChannelInterrupt(PIT_INST, CH_0);
  Pit_Ip_StartChannel(PIT_INST, CH_0, PIT_PERIOD);

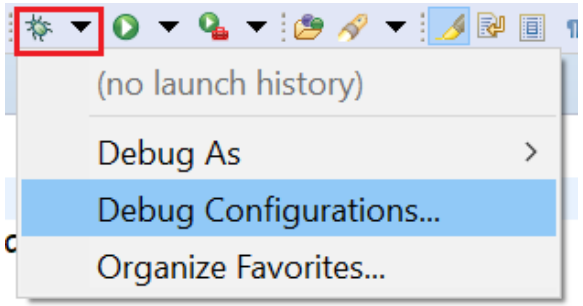
  for(;;) {}

return exit_code;
}
```

- Build the application (click the hammer icon).



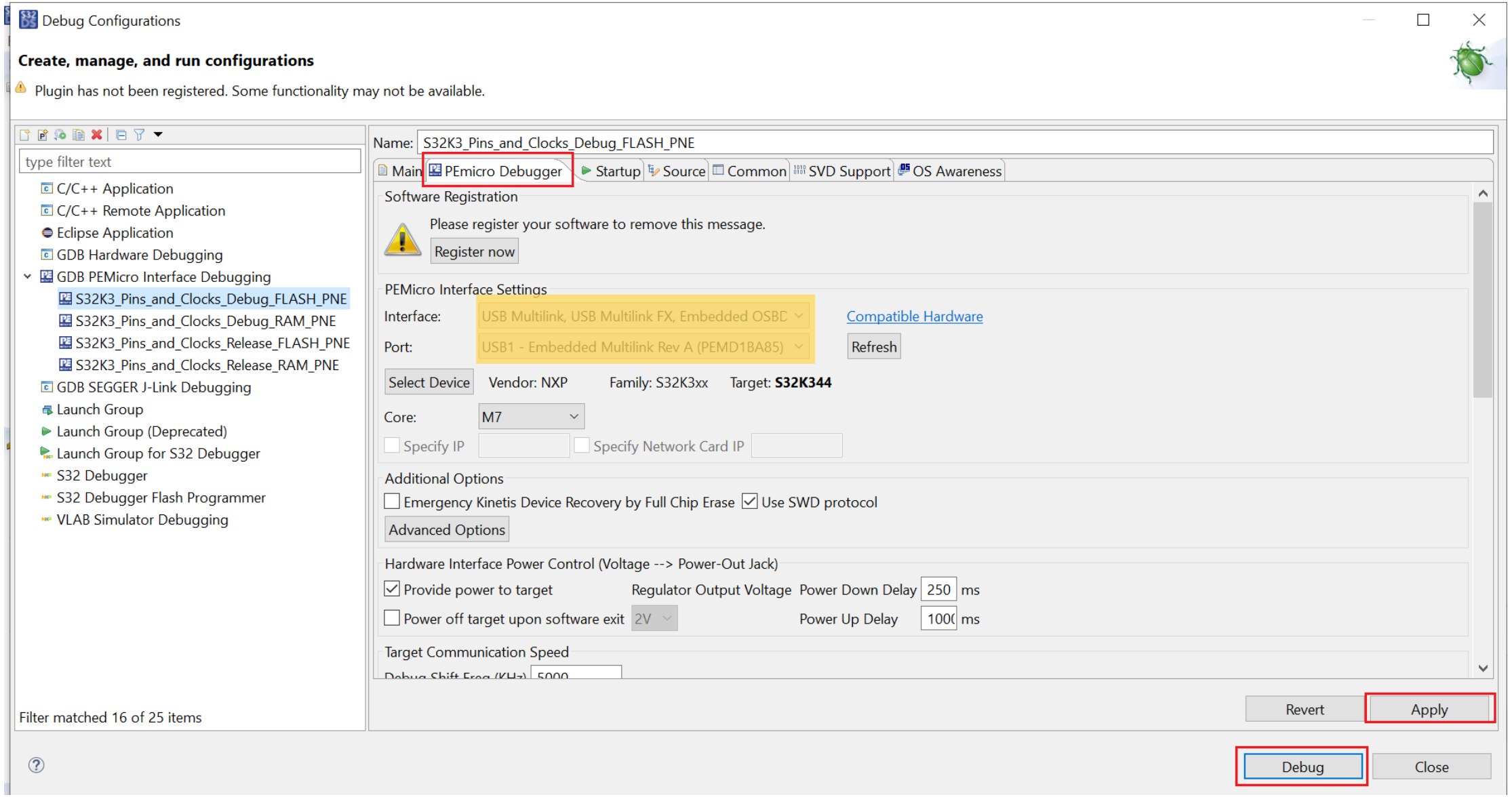
- Debug the application (click the bug icon).

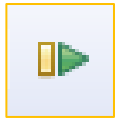


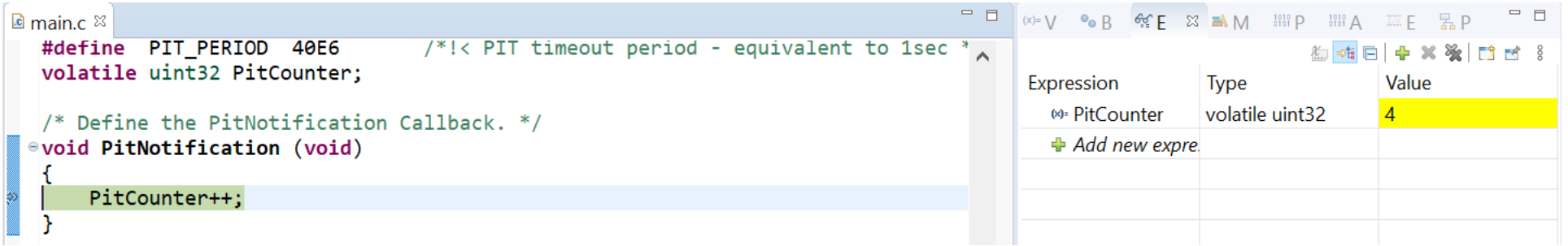
- Leave the standard debug configuration as it is. (Check the next slide to be sure)
- Click on **Apply** and then **Debug**

Before debugging your application, remember to have the board (EVB) connected

DEBUG THE APPLICATION



- Place a breakpoint within the **PitNotification** callback**.**
- Under the Expression Tab, add the **PitCounter** variable, so you can see its value change.
- Run de application (click the play icon). 



S32K3 Signal Multiplexing (SIUL2) Overview



SECURE CONNECTIONS
FOR A SMARTER WORLD



PUBLIC

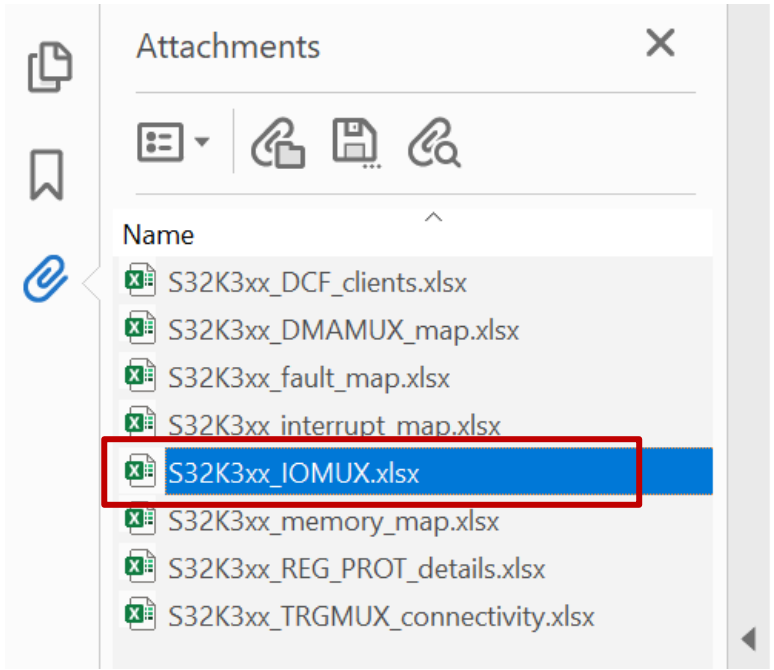
NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.

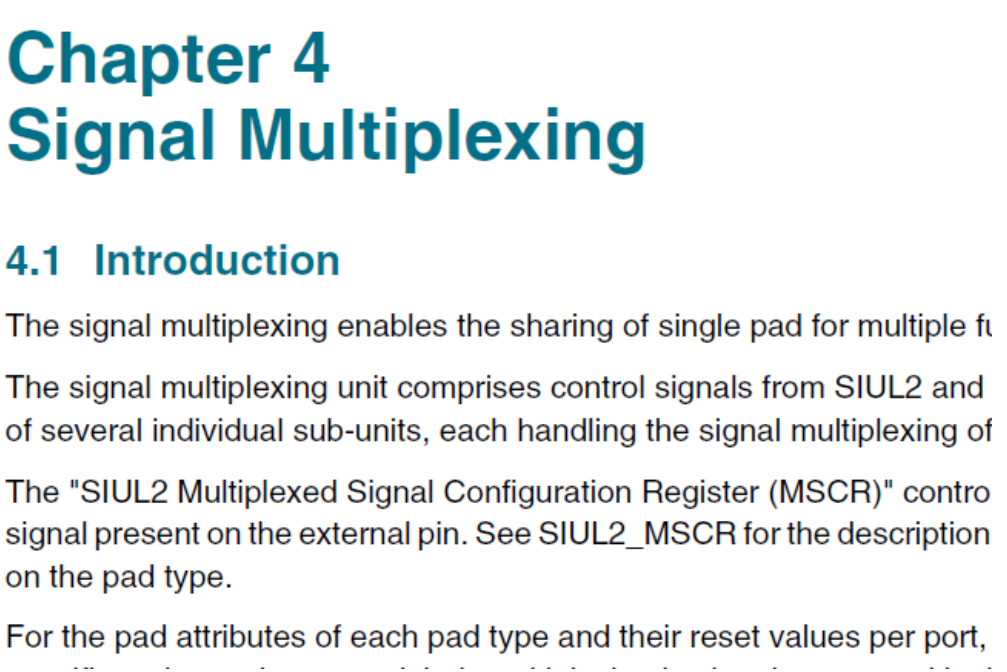
- There are up to **220** GPIOs in the S32K3x4, divided into **seven** port groups (PTA, PTB, PTC, PTD, PTE, PTF and PTG) and assigned to **one** I/O power domain (VDD_HV_A or VDD_HV_B).

- There are four pad types:

Pad type	Switching up to	High drive-strength	Slew-rate control
GPIO Standard	10 MHz	-	-
GPIO Standard plus	25 MHz	✓	-
GPIO Medium	50 MHz	✓	✓
GPIO Fast	120 MHz	✓	✓

- For the attributes of each pad, refer to the tab “IO Signal Table” the *S32K3xx_IOMUX.xlsx* attached to the RM.





Chapter 4
Signal Multiplexing

4.1 Introduction

The signal multiplexing enables the sharing of single pad for multiple fu

The signal multiplexing unit comprises control signals from SIUL2 and of several individual sub-units, each handling the signal multiplexing of

The "SIUL2 Multiplexed Signal Configuration Register (MSCR)" contro signal present on the external pin. See SIUL2_MSCR for the description on the pad type.

For the pad attributes of each pad type and their reset values per port,

The *S32K3xx_IOMUX.xlsx* columns are described below:

Column	Description
Port	PAD names of the device.
CR	Corresponding Multiplexed Signal Configuration Register (MSCR).
SSS	Alternative mode of operation.
Function	Functionality of the pad as per the corresponding Alternative mode (SSS).
Module	Module which is governing the pad.
Description	Short description of pad functionality.

The *S32K3xx_IOMUX.xlsx* columns are described below:

Column	Description
Direction	Pad direction (Input, Output or Input/Output).
Pad Type	Pad type (Standard, Standard plus, Medium or Fast).
Pin number in the supported packages for the device	
I/O Power Domain	Power domain of associated pad (VDD_HV_A / VDD_HV_B).
Pad states during and post different device resets.	
Default, reset values and the configurable bit fields of MSCR.	

- The SIUL2 provides control and configurability of the functions and electrical characteristics that appear on external device pins.

- S32K3xx PORT assignments are shown below

Port	SIUL2_MSCRn index
PortA[0-31]	MSCR0 - MSCR31
PortB[0-31]	MSCR32 - MSCR63
PortC[0-31]	MSCR64 - MSCR95
PortD[0-31]	MSCR96 - MSCR127
PortE[0-31]	MSCR128 - MSCR159
PortF[0-31]	MSCR160 - MSCR191
PortG[0-27]	MSCR192 - MSCR219

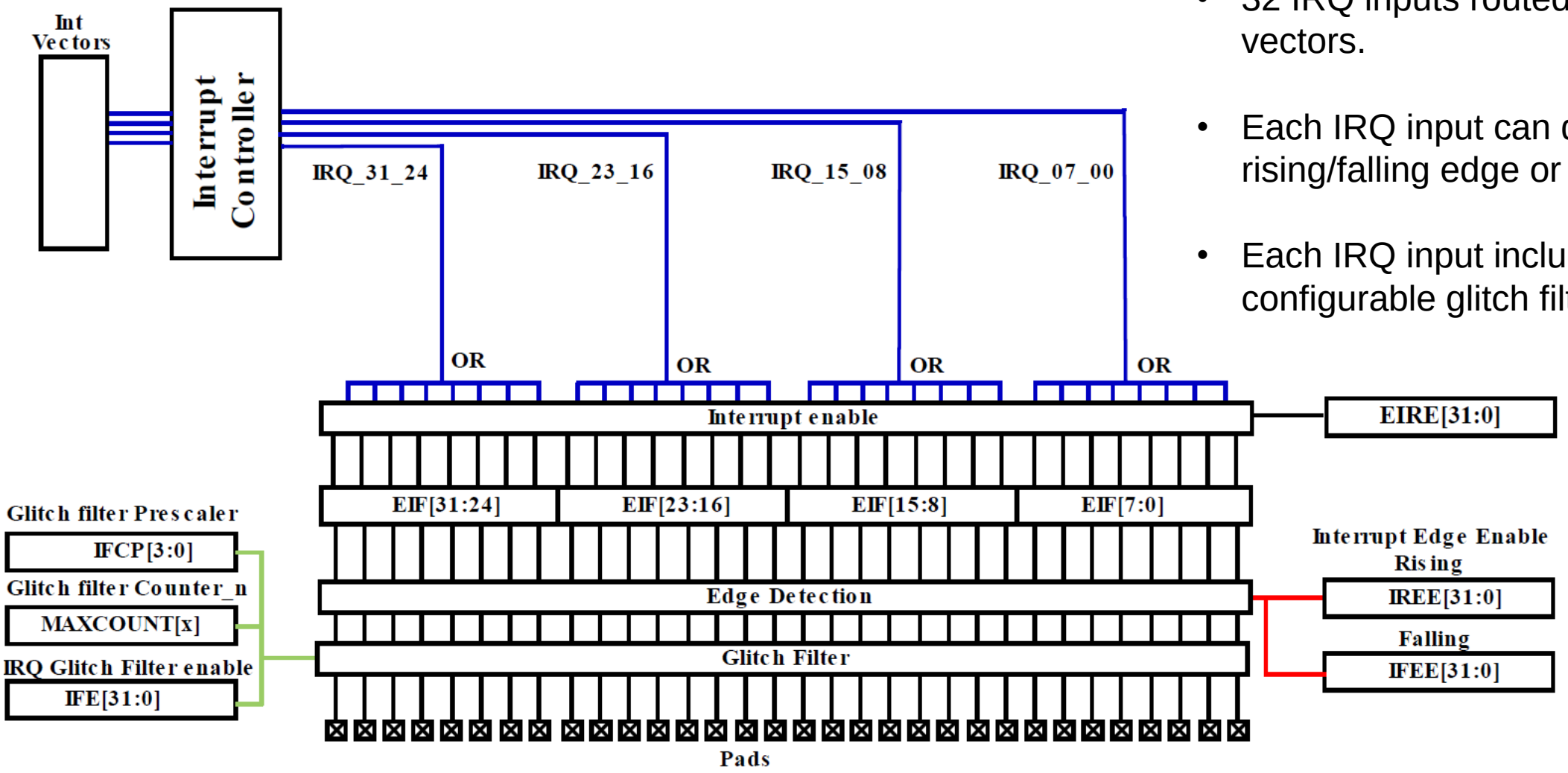
MSCR: Multiplexed Signal Control Register

- The system has four possible interrupt vectors available for the REQ pins in the SIUL2, the 32 interrupt request sources mapping is shown below.

Vector/Channel #	Interrupt vector source
0	REQ[07] | REQ[06] | REQ[05] | REQ[04] | REQ[03] | REQ[02] | REQ[01] | REQ[00]
1	REQ[15] | REQ[14] | REQ[13] | REQ[12] | REQ[11] | REQ[10] | REQ[09] | REQ[08]
2	REQ[23] | REQ[22] | REQ[21] | REQ[20] | REQ[19] | REQ[18] | REQ[17] | REQ[16]
3	REQ[31] | REQ[30] | REQ[29] | REQ[28] | REQ[27] | REQ[26] | REQ[25] | REQ[24]

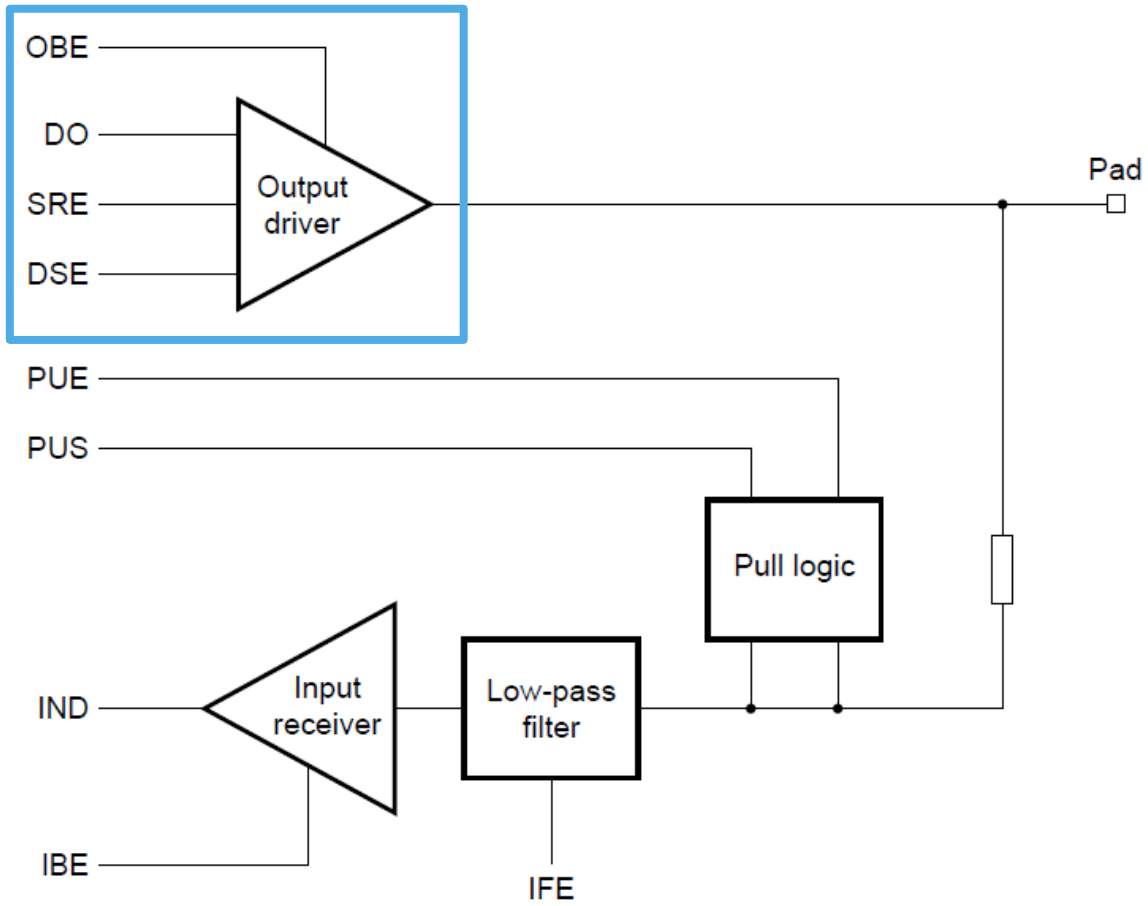
The REQ pins on the device map to independent **DMA** request channels.

EXTERNAL INTERRUPT REQUEST



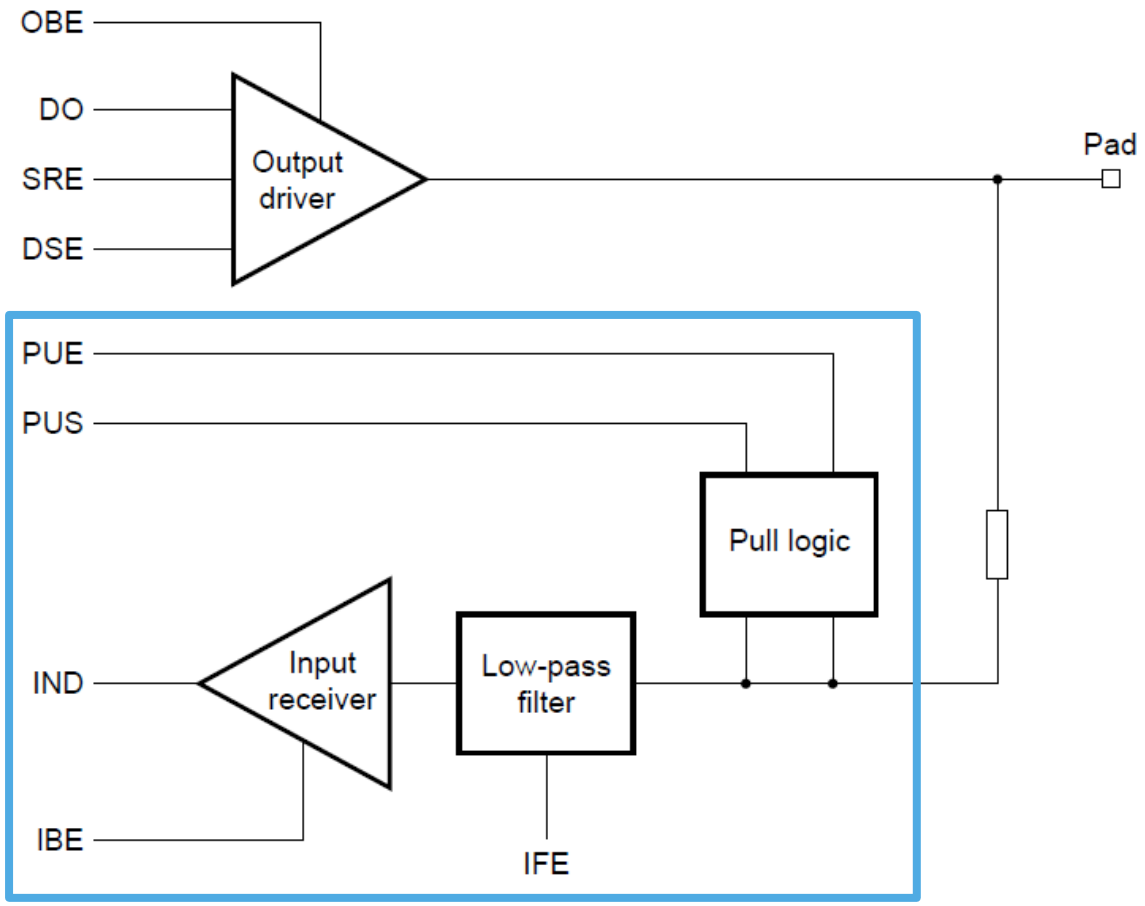
- 32 IRQ inputs routed to 4 Int vectors.
- Each IRQ input can detect rising/falling edge or both edges.
- Each IRQ input includes configurable glitch filter.

GPIO PAD



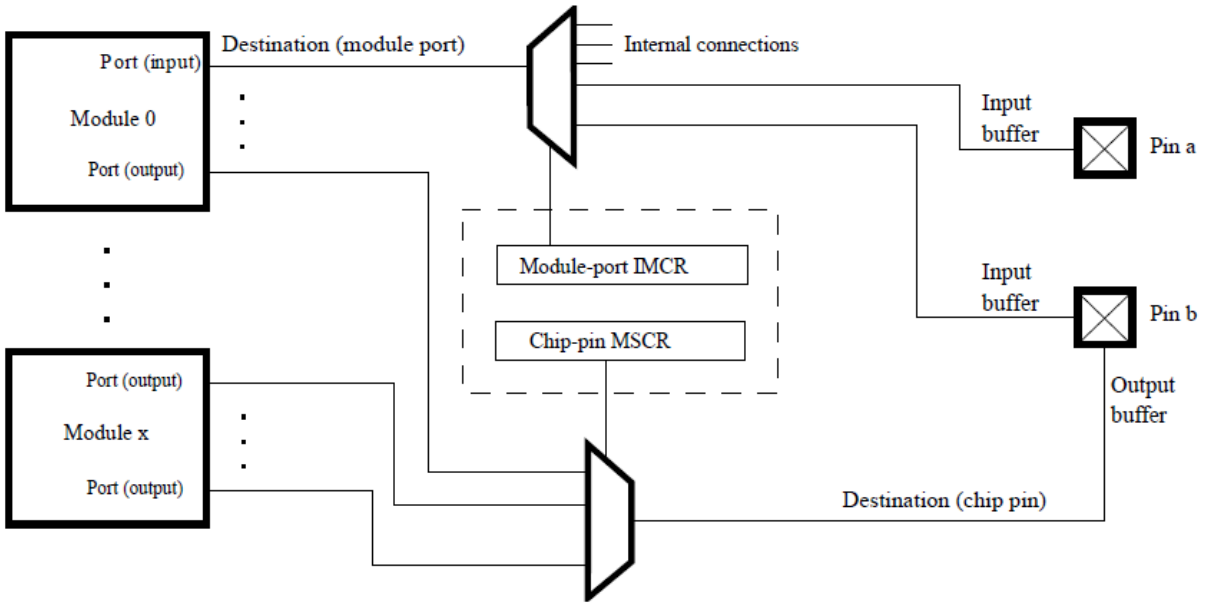
MSCR	
OBE	Controls the output buffer.
DO	Data coming from the core into the pad.
SRE	Enables Slew rate.
DSE	Enables Driver-strength.

GPIO PAD



MSCR	
PUE	Enables/Disables the internal pull resistor.
PUS	Selects pulldown or pullup.
IND	Data coming out of the pad into the core.
IBE	Controls the input buffer.
IFE	Enables/Disables input filtering.

- For input selection, there is another dedicated register: **IMCR**, Where the **SSS** field, must be configured according to the *S32K3xx_IOMUX.xlsx.*



- "CR" numbers 0 to 511 correspond to the MSCR register instances.
- "CR" numbers 512 to 1023 correspond to the IMCR register instances.

SW EXAMPLE: SETTING PTB0 AND PTB1 AS CAN0 RX AND TX

378	PTB0	SIUL_MSCR32	0000_0000	GPIO[32]	SIUL
388	PTB0	-	-	WKPU[7]	WKPU
389	PTB0	SIUL_IMCR512	0000_0011	CAN0_RX	CAN0
390	PTB0	SIUL_IMCR535	0000_0001	EIRQ[8]	SIUL
391	PTB0	SIUL_IMCR563	0000_0100	eMIOS_0_CH[3]_G	eMIOS_0
392	PTB0	SIUL_IMCR598	0000_0001	eMIOS_1_CH[6]_H	eMIOS_1
393	PTB0	SIUL_IMCR678	0000_0011	FXIO_D14	FXIO

```
// CAN0_RX (PTB0)
SIUL2->IMCR[512-512] = SIUL2_IMCR_SSS(0b011); // CAN0_RX
SIUL2->MSCR[32]      = SIUL2_MSCR_IBE_MASK; // Input Buffer
```

397	PTB1	SIUL_MSCR33	0000_0000	GPIO[33]	SIUL
398	PTB1		0000_0001	LPI2C0_SCLS	LPI2C0
399	PTB1		0000_0010	LPUART0_TX	LPUART0
400	PTB1		0000_0011	LPSPI0_SOUT	LPSPI0
401	PTB1		0000_0100	eMIOS_0_CH[7]_G	eMIOS_0
402	PTB1		0000_0101	CAN0_TX	CAN0

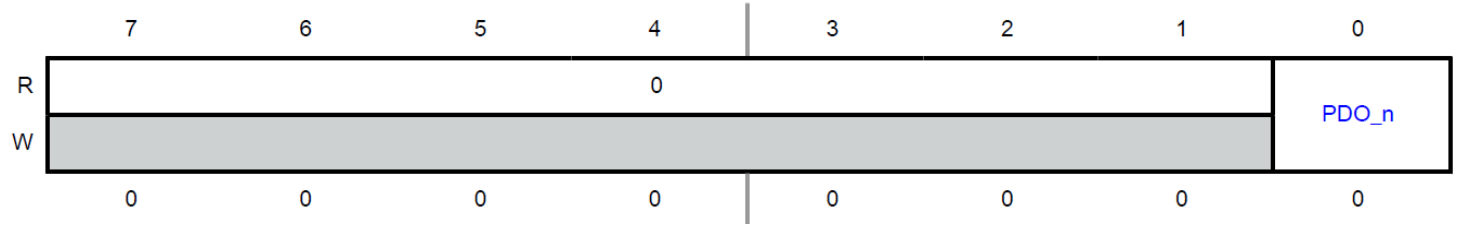
```
// CAN0_TX (PTB1)
SIUL2->MSCR[33] = 0b101 // CAN0_TX
                | SIUL2_MSCR_OBE_MASK; // Output Buffer
```

"CR" numbers (512 to 1023) correspond to the **IMCR** register instances.

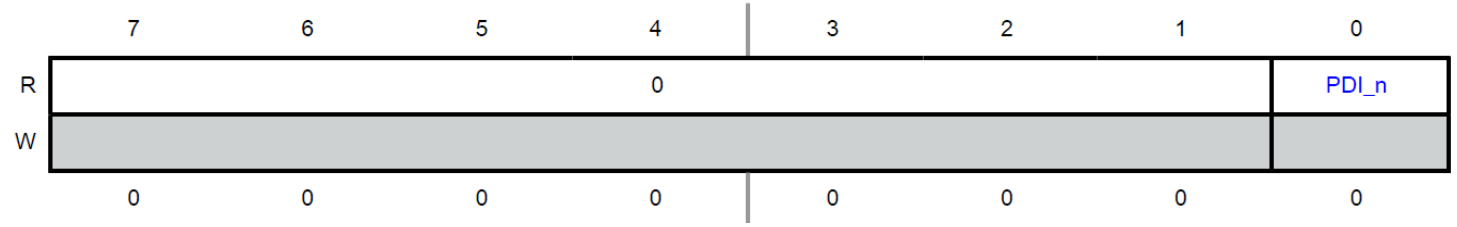
IMCR512 – 512 = **IMCR0**

- SIUL2 has separate data input and data output registers for all pads.
- Data output registers support both read and write operations.
- Data input registers support read access only.
- When a pad is configured to use one of its alternate functions, the data input values reflect the respective value of the pad and write operations do not affect the pad state.

GPDO0 - GPDO219



GPDI0 - GPDI219



Lab:
S32DS Pins ConfigTools

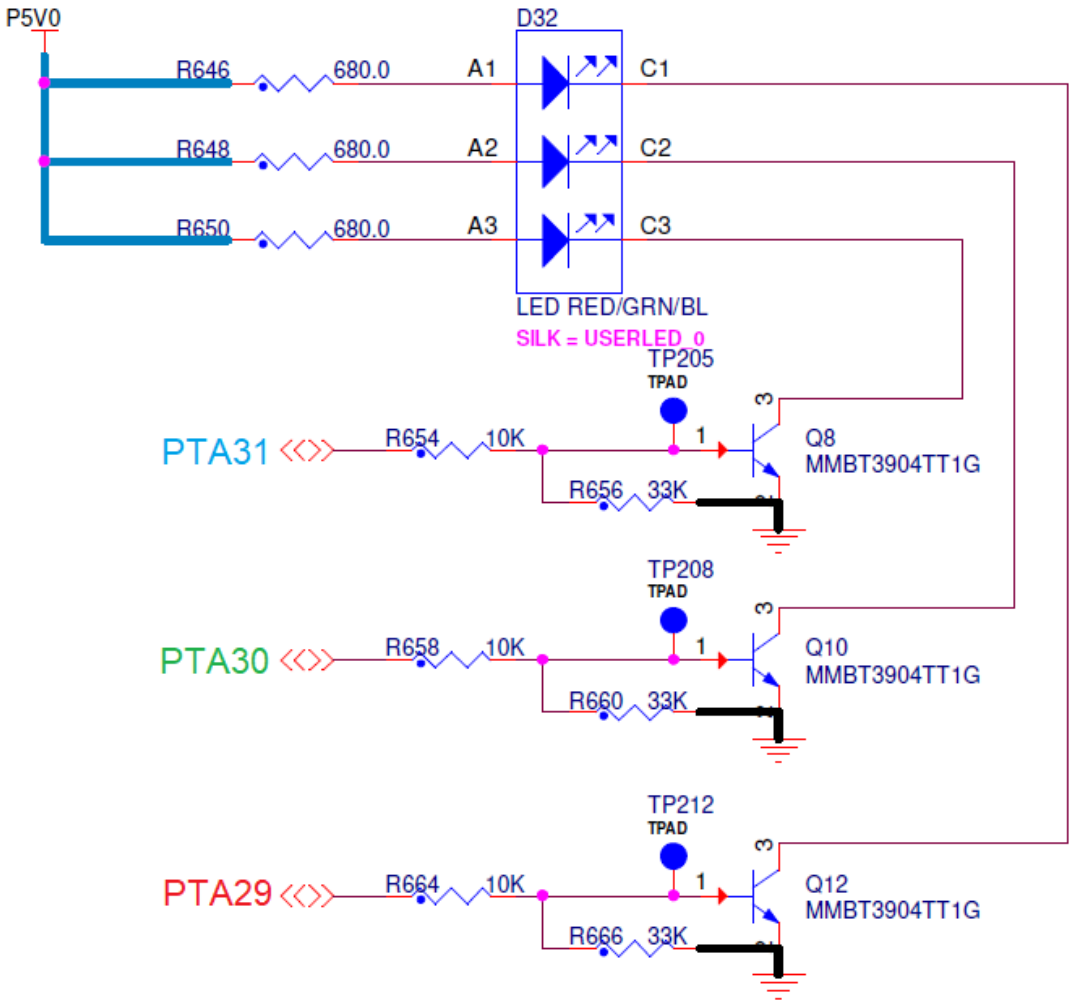
SECURE CONNECTIONS
FOR A SMARTER WORLD



PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.

OBJECTIVES

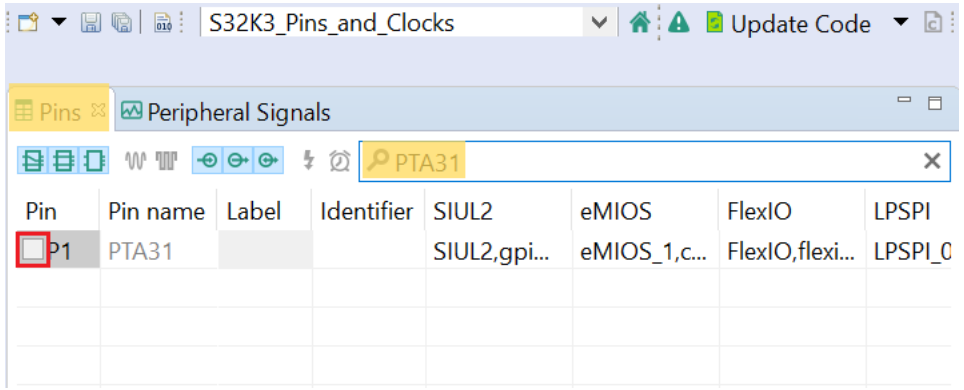


- Reuse the previous application project shown in the Lab: S32DS Clocks & Peripherals ConfigTools section
- Look at the S32DS Pins **CT**.
- Toggle a GPIO (**PTA31**) every PIT timeout (1 sec).

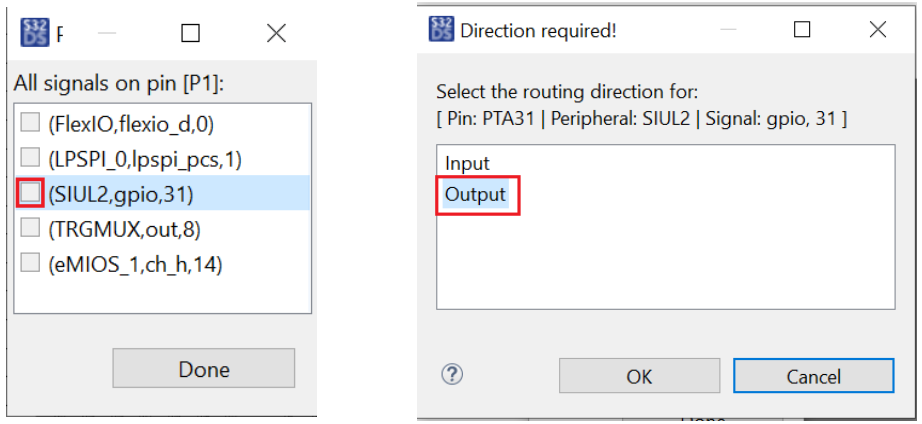
- Open the **Pins** Configuration Tool.



- In the **Pins** tab, type **PTA31** and click the corresponding pin name.

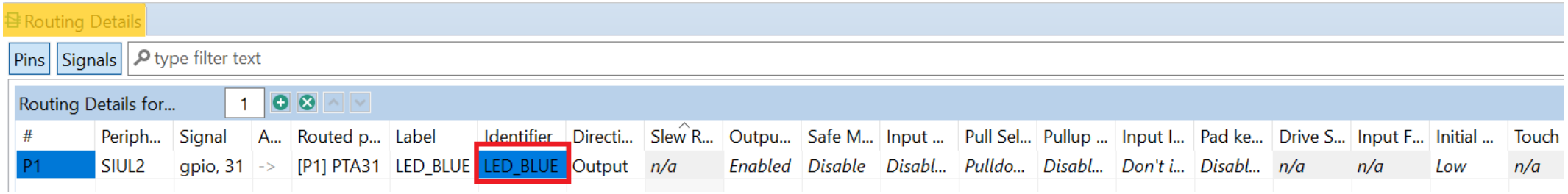


- Select the pin alternative (**SIUL2**) and direction (**Output**). Then click OK, Done.

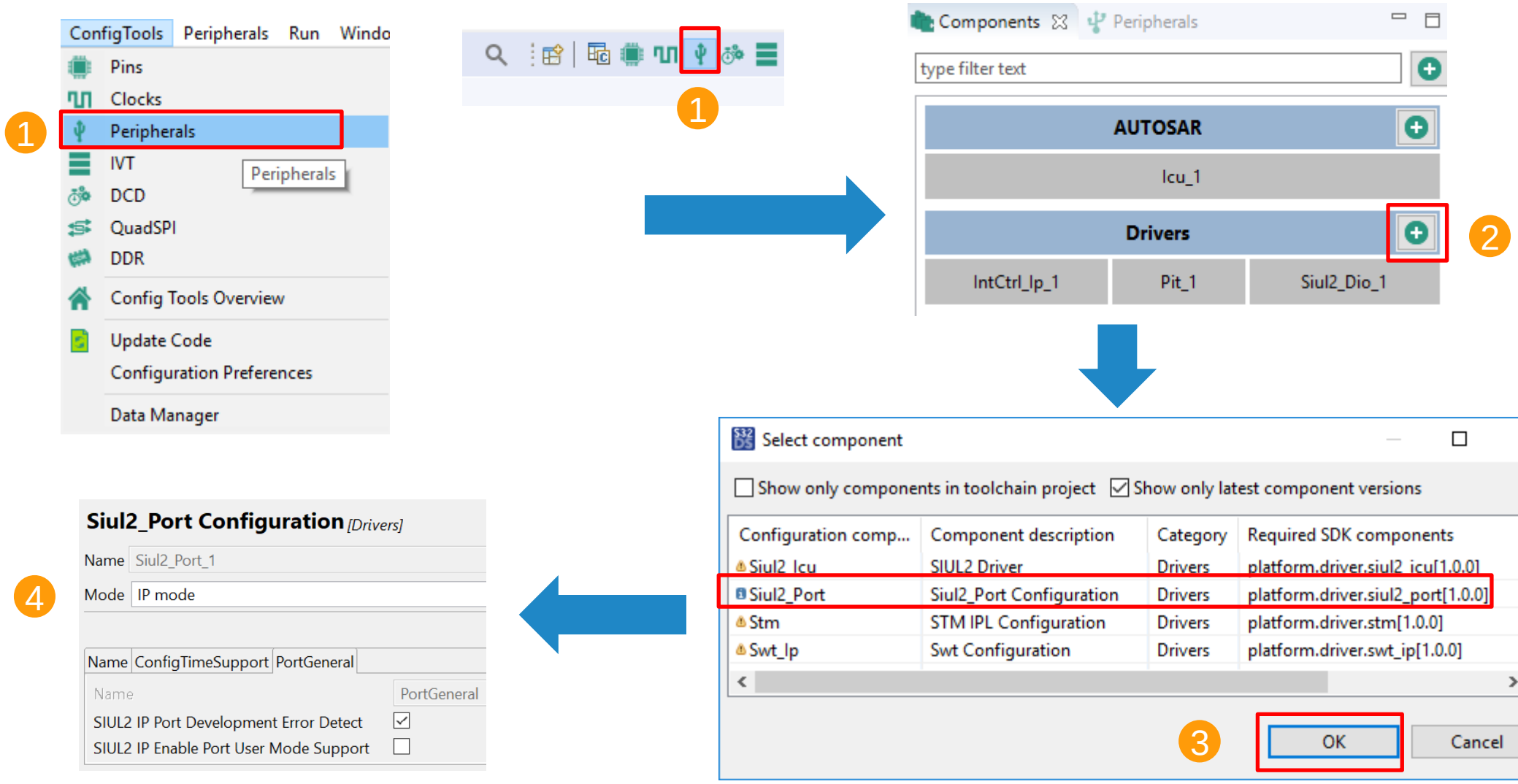


- In the **Routed Pins** tab, you can continue configuring the electrical characteristics of your pin, such as *pull resistors, drive strength or pad keeping*.

- It is recommended to use the **Identifier** field to give a meaningful ID to each pin.

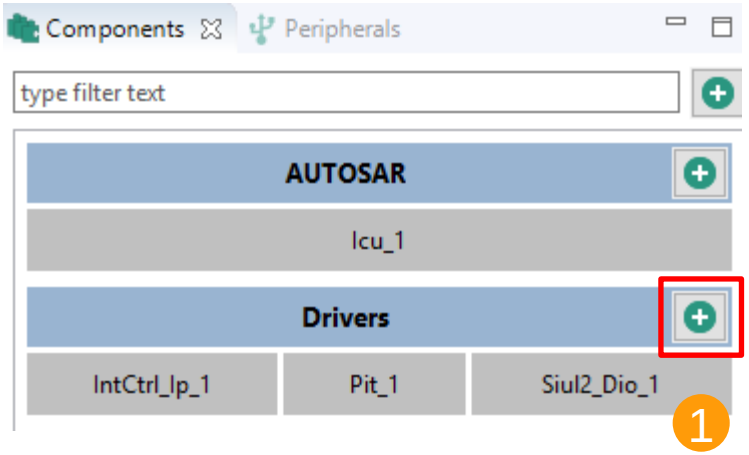


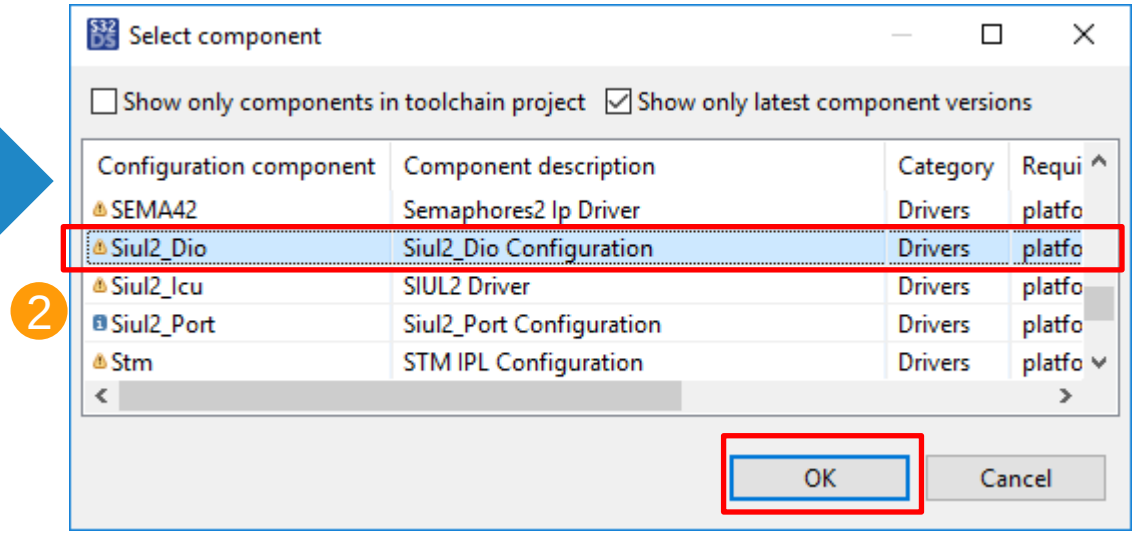
- After configuring all the desired pins. Go to the Peripherals view and add the Siul2_Port driver.

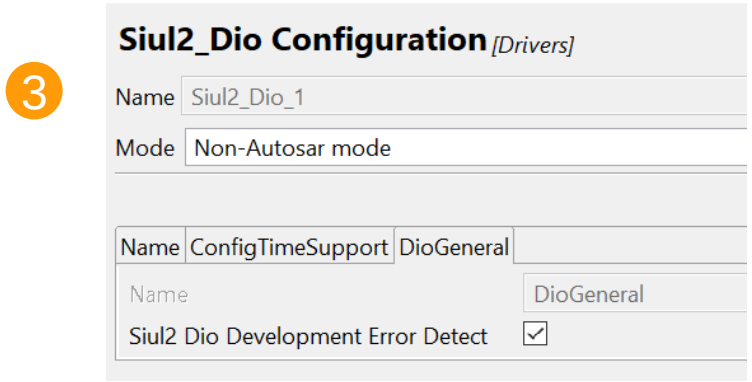


ADD SIUL2_DIO MODULE

- The Siul2_Dio driver has the necessary functions for digital input/output pins







- Add the generated Pins code to the application project.



- In the **main.c** include the SIUL2 header and add the following macros

```
#include "Siul2_Port_Ip.h"
#include "Siul2_Dio_Ip.h"

#define SIUL_INST       0U      /*!< SIUL2 instance used - 0 */
#define SIUL_GPDO_LED   31U     /*!< SIUL2 GPDO offset for PTA31 */
```

- Call the PORT initialization function, and Toggle PTA31 in the PIT callback as follows.

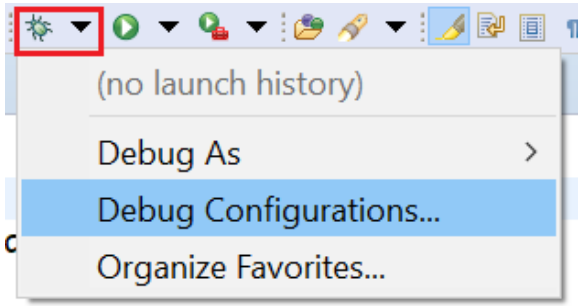
```
  /* Pins configuration */
  Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0, g_pin_mux_InitConfigArr0);

void PitNotification(void)
{
  PitCounter++;
  Siul2_Dio_Ip_TogglePins(LED_BLUE_PORT, (1<<LED_BLUE_PIN));
}
```

- Build the application (click the hammer icon).



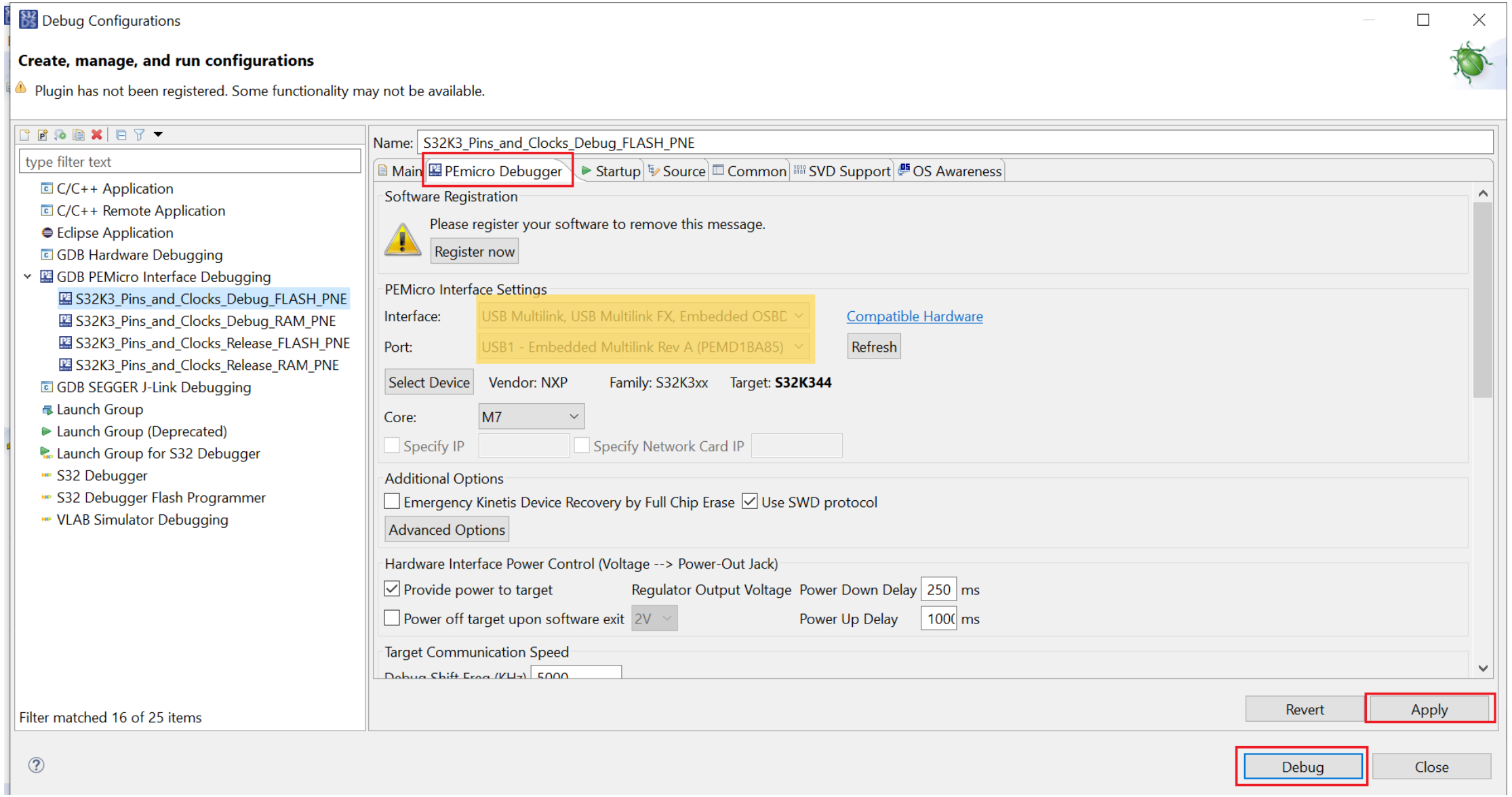
- Debug the application (click the bug icon).



- Leave the standard debug configuration as it is. (Check the next slide to be sure)
- Click on **Apply** and then **Debug**

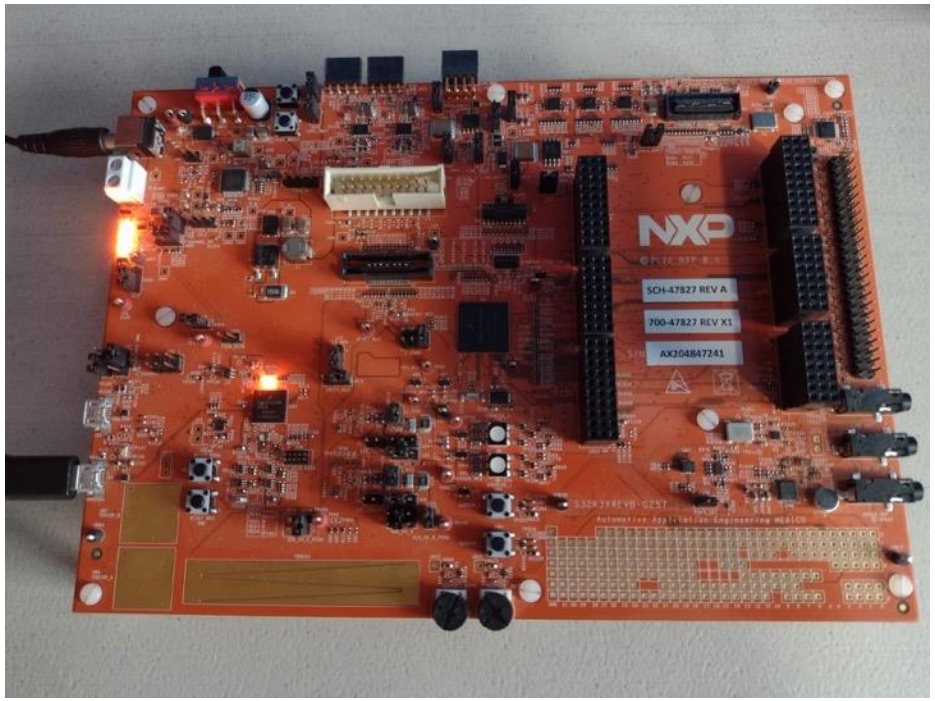
Before debugging your application, remember to have the board (EVB) connected

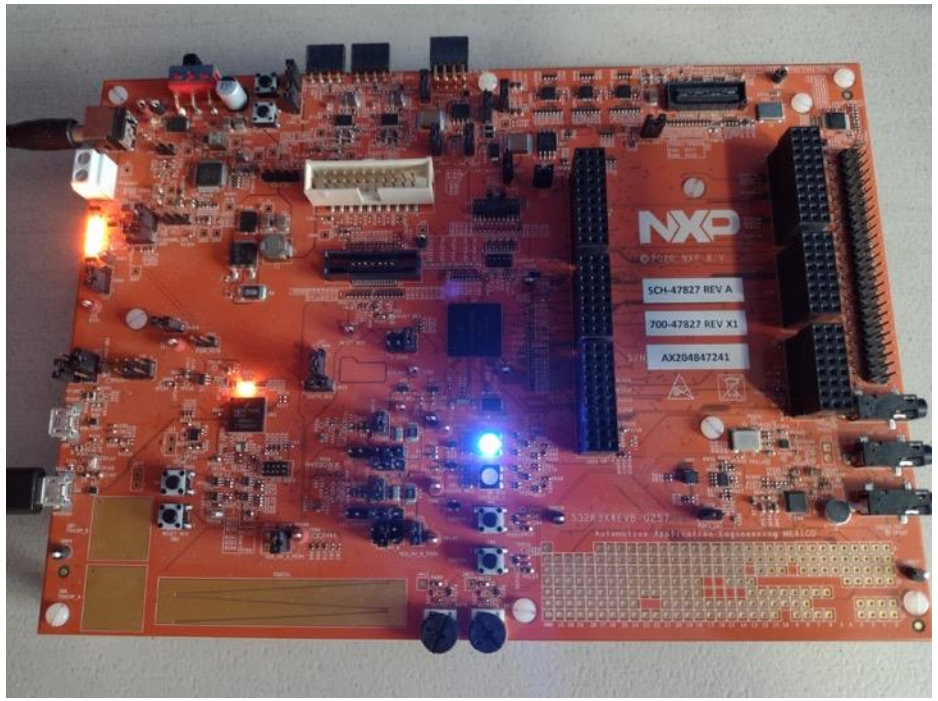
DEBUG THE APPLICATION



- Run de application (click the play icon). 

Observe the **BLUE LED** toggling every 1sec on the board.







S32K3 MICROCONTROLLERS FOR AUTOMOTIVE AND INDUSTRIAL

Tackling cost and complexity of automotive software development

S32K MCU family page

S32K3 MCU page

S32DS for S32 platform page

Online engineering support:

- S32K MCUs community
- S32DS community
- S32 Configuration Tool



SECURE CONNECTIONS
FOR A SMARTER WORLD

SHOWROOM.NXP.COM

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V. ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.